TU Clausthal

Bachelor Thesis

by

Steve Leonel, Yomi Mbiakop

Beyond Accuracy: Measuring Intelligence in Programming by Example

1st Supervisor: **Prof. Dr. Christian Bartelt**

2nd Supervisor: **Dr. Sascha Marton**

30th August 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

Hiermit versichere ich, dass ich die Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe und dass alle Stellen dieser Arbeit, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, als solche kenntlich gemacht wurden und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsstelle vorgelegt wurde.

Des Weiteren erkläre ich, dass ich mit der öffentlichen Bereitstellung meiner Abschlussarbeit in der Instituts- und/oder Universitätsbibliothek einverstanden bin.

Ich stimme hiermit außerdem einer softwaregestützten Plagiatsprüfung zu.

Clausthal-Zellerfeld, 30th August 2026	
Location, Date	Steve Leonel, Yomi Mbiakop

Abstract

Programming by Example (PBE) lets users who cannot write code build programs simply by providing a handful of input-output examples, powering tools from spreadsheet automation to LLM-based coding assistants. Such users typically cannot read the generated program, leaving them little way to judge whether it captures their intent beyond rechecking it against the examples already given. Current evaluation practice mirrors this blind spot: PBE approaches are almost always compared by accuracy, the fraction of tasks whose output exactly matches the target. But accuracy is itself ambiguous, since it can mean checking a program against a reference or an output against the expected one, and either way a task collapses into a single binary success signal (BSS) that cannot separate a near miss from a complete failure or explain why a solved task worked.

This thesis develops four token-based program metrics, the Program Operation, Position, Sequence, and Edit Score (POS, PPS, PSS, PES), comparing a generated program directly to a reference program rather than only outputs. They are applied to two PBE approaches with fundamentally different representations: DeepCoder's fixed domain-specific language and DreamCoder's typed lambda calculus, which grows its own library of reusable components through a wake-sleep cycle. A shared evaluation procedure and purpose-built normalization pipeline make these two otherwise incomparable representations measurable through a shared normalized operation vocabulary.

Across both approaches, the token-based metrics reveal structural detail the BSS alone cannot show: by construction, the POS never falls below the other three metrics, so the informative signal lies in how large this gap actually is, a gap that varies by approach, configuration, and metric rather than following a single fixed pattern. A task can be marked solved while its program bears little resemblance to the reference, and a task marked entirely wrong can already contain a structurally informative near-solution.

These findings show that a BSS can conceal substantial structural differences between programs receiving the same pass/fail outcome, allowing researchers to look beyond the binary verdict. Solving a task is no longer the only question worth asking; how it got there, and how far it still has to go, matter just as much.

Zusammenfassung

Programming by Example (PBE) ermöglicht es Nutzern, die nicht programmieren können, Programme allein anhand weniger Input-Output-Beispiele zu erstellen, von der Automatisierung in Tabellenkalkulationen bis zu modernen LLM-basierten Programmier-Assistenten. Solche Nutzer können das erzeugte Programm meist nicht selbst lesen, weshalb ihnen kaum eine Möglichkeit bleibt zu beurteilen, ob es ihre Absicht trifft, außer es erneut an denselben Beispielen zu prüfen, die sie bereits gegeben haben. Die gängige Evaluationspraxis spiegelt genau diesen blinden Fleck wider: PBE-Ansätze werden fast ausschließlich über accuracy verglichen, den Anteil der Aufgaben, deren Ausgabe exakt mit dem Ziel übereinstimmt. Accuracy selbst ist aber mehrdeutig, da sie entweder ein erzeugtes Programm gegen ein Referenzprogramm oder eine erzeugte Ausgabe gegen die erwartete prüfen kann, und so oder so schrumpft jede Aufgabe auf ein einziges binäres Erfolgssignal (BSS) zusammen, das einen knappen Fehlschlag nicht von vollständigem Versagen unterscheidet und nichts darüber aussagt, warum eine gelöste Aufgabe funktioniert hat.

Diese Arbeit entwickelt vier tokenbasierte Programmmetriken, den Program Operation, Position, Sequence und Edit Score (POS, PPS, PSS, PES). Statt nur Ausgaben zu vergleichen, stellen sie ein erzeugtes Programm direkt einem Referenzprogramm gegenüber, angewendet auf zwei PBE-Ansätze mit grundverschiedener Programmrepräsentation: DeepCoders feste domänenspezifische Sprache (DSL) und DreamCoders typisierten Lambda-Kalkül, der seine eigene Bibliothek wiederverwendbarer Bausteine über einen Wake-Sleep-Zyklus erweitert. Ein einheitliches Evaluationsverfahren und eine eigens entwickelte Normalisierungspipeline machen diese beiden sonst unvergleichbaren Programmrepräsentationen auf demselben Token-Vokabular messbar.

Bei beiden Ansätzen zeigen die tokenbasierten Metriken strukturelle Details, die dieses BSS allein nicht sichtbar machen kann: Der POS kann konstruktionsbedingt nie niedriger liegen als die drei übrigen Metriken, das informative Signal liegt daher darin, wie groß diese Lücke tatsächlich ausfällt, eine Lücke, die je nach Ansatz, Konfiguration und Metrik unterschiedlich groß ist, statt einem einzigen festen Muster zu folgen. Eine Aufgabe kann

als gelöst gelten, obwohl ihr Programm kaum dem Referenzprogramm ähnelt, und eine als vollständig falsch bewertete Aufgabe kann bereits eine strukturell aufschlussreiche Beinahe-Lösung enthalten.

Diese Befunde zeigen: Ein BSS kann erhebliche strukturelle Unterschiede zwischen Programmen mit demselben binären Ergebnis verbergen und ermöglicht es Forschenden und Praktikern dadurch, über das reine Bestehen-oder-nicht-Urteil hinauszublicken. Ob eine Aufgabe gelöst wurde, ist damit nicht mehr die einzige Frage, die zählt; wie ein Ansatz dorthin gelangt ist und wie weit er noch entfernt ist, zählt genauso.

Acknowledgements

I would like to thank my two examiners, Prof. Dr. Christian Bartelt and Dr. Sascha Marton, for evaluating this thesis.

I am especially grateful to my supervisor, Janis Zenkner, for his guidance and feedback throughout the course of this work.

A heartfelt thank you goes to my parents for their unconditional support.

I would also like to thank Dr. Leon Kellerhals, who supported me greatly throughout my studies.

I would also like to thank all the other professors at TU Clausthal who accompanied me throughout my studies and shared their valuable knowledge with me.

I also thank my friends Jonathan Nguemnang and Josias Gapinssi, who read through my thesis and gave me valuable feedback while writing it.

Last but not least, I thank my entire family and all my friends for their support and patience throughout my studies.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation

Programming by Example (PBE) is an approach to program synthesis (PS) in which a desired program is derived from a set of input-output examples [10, 2]. Instead of writing a program manually or formulating a formal specification, the user specifies the desired behavior through examples [9]. A PBE approach then attempts to find a program that satisfies these examples and generalizes to new inputs [10, 16]. This makes PBE especially relevant for automating tasks such as data cleaning, string transformations, and the processing of tabular data, since it allows users without advanced programming skills to specify what they want through a few examples rather than writing code [14, 9, 2].

A widely used example of PBE in practice is Microsoft Excel's Flash Fill feature, which infers a string-transformation program from a handful of example cells the user has filled in by hand, for instance splitting a full name into separate first- and last-name columns, and then applies the inferred program to the remaining rows [9]. Flash Fill illustrates both the appeal and the risk of PBE at once: the user only ever sees the transformed values, not the program that produced them, so a plausible-looking but subtly wrong transformation can go unnoticed until it silently corrupts a row the user never checked by hand.

PS restricts the space of candidate programs it searches through a domain-specific language (DSL), a language that fixes which operations, constants, and program structures are permitted for a given class of tasks [9, 2]. PBE is the special case of PS in which the specification takes the form of input-output examples rather than, say, a formal condition or natural-language description.

PBE approaches can be realized in different technical ways: classical approaches are frequently based on symbolic search in a DSL, while more recent approaches also include neural and Large Language Model (LLM)-based models [10, 5, 16]. Regardless of the concrete synthesis approach, however, the central question remains the same: whether

the generated program actually captures the intended transformation, or merely happens to satisfy the examples it was given. This question matters in practice, because PBE approaches are increasingly aimed at users who cannot inspect the generated program themselves and must trust the approach's output at face value [14, 9].

This makes evaluation a fundamental problem in PBE: satisfying the provided input-output examples does not necessarily mean that the generated program captures the intended transformation.

This issue is directly reflected in how the performance of PBE approaches is evaluated in practice.

1.2 Problem Statement

Many existing evaluation approaches assess PBE approaches using accuracy metrics. Typically, this checks whether the output produced by the approach exactly matches the expected output [16]. If this is the case, the task is considered correctly solved; otherwise it is judged incorrect [3, 16].

This binary evaluation is simple and intuitive, but it falls short in many cases. A generated solution can be partially correct even if it does not exactly match the expected output. Suppose, for example, that an approach is meant to transform the input `[3, 1, 2]` by doubling each element and sorting the result, so that the expected output is `[2, 4, 6]`. If the approach instead produces `[3, 4, 5]`, having correctly sorted the input but added 2 to each element instead of doubling it, this binary judgment marks the solution as completely wrong. At the same time, the output shows that the approach has captured part of the intended transformation: it identified that the input needed to be sorted and applied some operation to each element, just not the correct one.

As the example illustrates, this binary success signal (BSS) reduces every solution to a single judgment and cannot express how close a wrong solution actually is to the target. An arbitrary, unrelated program and a program that gets almost everything right receive the exact same score. This makes it difficult to tell whether an approach fails because it misunderstands a task entirely or because it comes structurally close to the correct solution but makes a small, systematic mistake.

This limitation is not confined to a single sorting example. Two generated programs can produce the same correct output on the observed examples while arriving at it in entirely different ways, one of them explicitly reproducing the intended transformation, the other combining several unrelated operations that happen to cancel out on exactly

the examples it was tested against. An evaluation that only inspects the output cannot distinguish these two cases, because it never looks at the program that produced it. The same asymmetry applies in the other direction: a program can be judged entirely wrong on the output level while it already contains most of the correct transformation, missing it only by a small, identifiable step. In both directions, whether an approach genuinely captured the intended rule or merely got lucky, and how close a failed attempt actually came, are questions that an output-level judgment by construction cannot answer.

The comparisons above are therefore possible at two different levels. One can compare the produced output to the expected output, as above, or one can compare the generated program itself to a reference program that is known to produce the correct behavior. Comparing at the program level makes it possible to see *how* an approach arrived at its result, not only whether the final output happens to be right, and can reveal partially correct or structurally close solutions that an output-level judgment alone would report as uniformly wrong. This thesis builds on exactly this additional perspective.

1.3 Aim of the Thesis

The aim of this thesis is to gain additional insight into the performance of PBE approaches beyond a BSS. To this end, it develops and evaluates metrics that measure how structurally close a generated program is to a reference program, at the level of the program itself rather than only its output. Instead of asking only whether a task was solved, such metrics ask how much of the reference program's token structure is preserved in the generated program and in what respect the two programs differ. Such a program-level comparison is not free of trade-offs: it requires a reference program to compare against and depends on how differently two approaches represent programs in the first place.

The thesis focuses on partial correctness as the central evaluation aspect that follows from this gap: whether a generated program, while not fully correct, nonetheless contains meaningful parts of the expected solution. This is intended to enable a more fine-grained evaluation of PBE approaches, one that can show in which cases a BSS provides only an incomplete picture of an approach's actual performance.

Condensed into a single, precisely formulated question, this aim becomes the starting point for the entire thesis.

1.4 Research Question

The central research question of this thesis is as follows:

> **What additional insights into the performance of PBE approaches can be gained through evaluation metrics beyond binary success signals?**

To answer this research question, the following aspects are considered in particular:

- Which aspects of PBE approach behavior remain hidden by a BSS?
- What additional information can be gained through the analysis of partial correctness?
- In which cases do program-level metrics provide a more informative explanation of why a task was solved or not solved?

To answer these questions, this thesis makes several concrete contributions.

1.5 Contribution of the Thesis

The contributions of this thesis are as follows:

- We adapt established concepts from edit-distance and sequence-similarity metrics to tokenized *programs* rather than their outputs, enabling structural similarity to be compared across different PBE approaches after representation-specific normalization.
- We demonstrate the transferability of this approach on two PBE approaches with fundamentally different program representations. Since a direct token comparison between DeepCoder's DSL and DreamCoder's lambda calculus would otherwise not be possible, a purpose-built mapping onto a shared vocabulary was required for this.
- We quantify structural differences between generated and reference programs that remain invisible to a BSS, including partial overlap on unsolved tasks and structurally divergent programs on tasks that satisfy all evaluation examples.

- We show that even failed search attempts can contain usable information, by extending DeepCoder's search so that a score beyond the value 0 becomes possible for a subset of unsolved tasks as well.

Realizing these contributions requires building up the necessary background first. Chapter 2 situates the thesis within existing approaches to solving and evaluating PBE tasks to clarify what specifically remains unaddressed. Chapter 3 then establishes the theoretical foundations of PS, PBE, and evaluation metrics on which the rest of the argument builds. Chapter 4 first describes how DeepCoder's and DreamCoder's programs are mapped onto a shared vocabulary despite their differing representations, and then introduces the four token-based metrics developed to close the identified gap. Chapter 5 fixes the concrete conditions under which both approaches are compared. Chapter 6 presents and discusses the resulting experiments together, including the limitations of this work. Chapter 7 summarizes the findings, answers the research question, and outlines directions for future work.

2 Related Work

Where this thesis fits into the existing literature follows from what it deliberately does not do: propose a new solving methodology. Instead, it investigates whether token-based metrics give more insight into an approach's performance than its BSS alone, so the literature below is organized along exactly this split: existing approaches are first grouped by *how* they solve PBE tasks, then by *how* they evaluate the resulting solutions, before DeepCoder and DreamCoder, the two PBE approaches to which the metrics developed in this thesis are applied, are introduced in detail.

2.1 Solving Programming by Example Tasks

Approaches to solving PBE tasks differ mainly in how they guide the search for a satisfying program. Some rely on predefined rules and decomposition, others on neural predictions or on feedback from executing partial programs, and a more recent line simply prompts a pretrained language model directly. Table 2.1 summarizes this by contrasting how a representative approach from each group represents programs and what guides its search, before the remainder of this section characterizes each in more detail.

Classical Search-Based Approaches. Gulwani et al. [10] give a general overview of PS and PBE. A well-known practical instance is FlashFill, which derives string transformations for spreadsheet columns from a handful of examples [9]. FlashMeta generalizes this idea into a reusable synthesis framework: it decomposes the specification of an output into specifications for its sub-expressions, so that domain-specific synthesizers for many string, format, and layout transformations can be constructed from shared building blocks rather than written from scratch [19]. Building on this line of work, FlashFill++ extends the underlying DSL to cover substantially more realistic spreadsheet transformations while achieving substantially faster synthesis than the original FlashFill in the reported evaluation [2]. Programming by Demonstration is a closely

Table 2.1: Representative PBE-solving approaches, contrasted by how they represent programs and what guides their search.

Approach	Program representation	Guidance mechanism
FlashMeta [19]	Domain-specific DSLs (e.g. FlashFill, FlashExtract)	Specification decomposition via witness functions, no learned component
RobustFill [5]	DSL (string transformations)	Seq2seq network generates the full program directly
DeepCoder [1]	Fixed DSL (list processing)	DFS guided by a neural attribute predictor
Write-Execute-Assess [6]	DSL (graphics, string editing)	Policy and value networks guided by execution state
DreamCoder [7]	Evolving typed lambda-calculus library	Wake-sleep cycle: library learning and neural recognition
Codex [3]	General-purpose Python, no DSL	Code-fine-tuned pretrained LLM, repeated sampling (pass@k)

related paradigm: Lau et al. [14] show that programs can also be derived from a user's demonstrated actions rather than explicit input-output pairs, representing the entire set of programs consistent with the demonstrations so far as a version space that is updated incrementally as more demonstrations arrive.

Neurally Guided Search. RobustFill trains a sequence-to-sequence model to predict a complete program directly from a set of input-output examples [5]. Devlin et al. compare four model variants that progressively add attention between the input-output encoder and the program decoder, finding that the attention-based variants substantially outperform a non-attentional baseline and, notably, remain robust when the given examples contain noise such as typos. DeepCoder instead keeps the search itself unchanged and uses a neural network only to predict, from the given examples, which operations of a fixed DSL a solving program is likely to use [1].

Execution-Guided and Decomposition-Based Synthesis. A different line of work focuses less on the search space itself and more on how the synthesis process unfolds step by step. Ellis et al. [6] equip a neural search process with a read-eval-print loop that immediately executes partially written programs, so that a policy model proposing the next piece of code and a value model assessing the partial program's prospects can both condition on the resulting execution state rather than on syntax alone. This addresses a basic difficulty of program search: small syntactic changes can produce

large, unpredictable semantic changes, which a purely syntactic search cannot anticipate. Building on this idea of decomposing synthesis into smaller steps, Shi et al. [24] propose ExeDec, which interleaves two neural models within a beam search over execution states rather than over raw code: a subgoal model that predicts the intermediate program state the next part of the program should reach, and a synthesizer model that generates code toward that predicted state. Zenkner et al. [30] revisit this framework and compare it against REGISM, their own ablated variant that relies solely on repeated execution-guided synthesis without explicit decomposition. Explicit decomposition helps with generalizing to longer programs and novel concept combinations, but the ablated, purely execution-guided variant frequently matches or even exceeds the decomposition-based approach on other measures. Decomposition, in other words, may not be the primary driver; iterative execution guidance likely accounts for much of ExeDec's performance instead. DreamCoder, the second approach evaluated in this thesis, similarly grows its own library of reusable program components across repeated learning cycles [7].

Program Synthesis using LLMs. Chen et al. [3] investigate, with Codex, how well large language models generate Python functions from natural-language docstrings, evaluated via the *pass@k* metric detailed in Section 2.2. A single sample from their 12-billion-parameter model solves 28.8% of their HumanEval benchmark, while sampling 100 candidates per problem and keeping any one that passes raises this to 70.2%. A model, it turns out, can often find a correct solution without reliably producing it on the first attempt. Li and Ellis [16] examine specifically whether LLMs solve classic PBE tasks, that is, tasks specified purely through input-output examples without any accompanying natural-language description. Their results show that LLMs can reach substantially higher performance after fine-tuning, particularly when test tasks are in-distribution, while performance degrades on out-of-distribution tasks, including cases involving unfamiliar combinations of operations not seen during training. More recently, Wei et al. [27] introduce CodeARC, an interactive benchmark in which agents iteratively refine candidate programs based on execution feedback rather than being judged on a single static attempt; even under this more informative evaluation, its best-performing model, o3-mini, solves only 52.7% of tasks on CodeARC's harder, name-anonymized split (59.5% on the easier annotated split). This suggests that the difficulty Li and Ellis [16] observed cannot be attributed solely to one-shot evaluation.

The guiding mechanism varies across all four groups. What stays constant is the criterion for judging the result: almost every approach discussed above is compared to

others by whether its output matches the expected outcome.

DeepCoder and DreamCoder, the two approaches evaluated in this thesis, come from two different groups among these and represent two structurally different approaches to solving PBE tasks. The metrics developed in this thesis can therefore be tested across different solving paradigms rather than only a single search mechanism. How well an approach can even be assessed on this basis, however, depends on how it is evaluated.

2.2 Evaluating PBE and Code-Generation Approaches

Existing evaluation approaches for PBE and code generation differ in how much they extract beyond a per-task pass/fail verdict. Some stick to purely functional correctness, others examine failure behavior in more targeted ways, and a third group compares the generated artifact itself rather than only its execution result.

Functional-Correctness Based Evaluation. Chen et al. [3] popularize *pass@k* for code generation: a problem counts as solved if at least one of k independently sampled programs passes the given test cases. Because it only requires one success among several attempts, pass@k separates a model's raw generative capability from the quality of a later selection step, but it says nothing about the many attempts that fail. APPS evaluates generated code against held-out test cases in the same functional-correctness spirit, additionally reporting an *Accuracy with Partial Credit* variant that counts the proportion of test cases passed per task rather than requiring all of them [11]. Hendrycks et al. show the practical relevance of this distinction empirically: for GPT-2 1.5B, plain accuracy, which requires all test cases to pass, averages 2.08%, whereas accuracy with partial credit, the average fraction of test cases passed per problem, reaches 9.00%. The substantially higher partial-credit score shows that binary accuracy masks functional progress on individual test cases even when a complete solution is not obtained. Compute-matched evaluation is a related concern: Sesterhenn et al. [23] show that reported performance differences between approaches can partly be explained by unequal compute budgets rather than by the approaches themselves. Reporting what was measured is not enough for a fair evaluation; the resource conditions under which it was measured matter too.

Beyond a Single Aggregate Score. Several works argue that even graded functional-correctness scores obscure important failure patterns. CodeBLEU combines n-gram

similarity with syntactic and semantic code features to address specific weaknesses of BLEU and exact match when applied to code [21]. CheckList takes a complementary, behavioral-testing perspective, showing across several NLP tasks that specific, systematic failure modes remain invisible when only a single aggregated accuracy value is reported [22]. Most recently, RACE evaluates generated code along four separate dimensions, readability, maintainability, correctness, and efficiency, and finds that even advanced LLMs satisfy customized, real-world requirements far less reliably than plain correctness scores would suggest [31]. These works motivate reporting more than one number per approach.

Sequence- and Code-Similarity Metrics. A separate line of work measures how close an incorrect output is to the target rather than only whether it is exactly right. Edit-distance metrics, today commonly referred to as Levenshtein distance after Levenshtein [15]'s work on binary codes, measure the minimum number of insertions, deletions, and substitutions needed to transform one sequence into another; Damerau [4] additionally considers transpositions of adjacent characters as a common class of typing errors. Longest Common Subsequence-based methods measure structural similarity differently: Lin and Och [17] use longest-common-subsequence overlap to score machine-translation output. Chapter 4 adapts this broader sequence-overlap idea but deliberately requires the matching token sequence to be *contiguous*, to make visible which uninterrupted parts of a program's structure are preserved. Further established metrics operate at yet other levels: BLEU scores n-gram precision against one or more references [18], and Kendall's coefficient scores rank correlation between two orderings [13]. Closer to program synthesis specifically, Song et al. [25] use Abstract Syntax Tree edit distance to compare generated and ground-truth code at the parsed-program level, capturing structural information that token-overlap metrics such as BLEU do not directly represent.

Existing code-similarity metrics such as CodeBLEU and AST-based edit-distance measures already compare generated code with reference code. The gap addressed in this thesis is narrower: it develops an interpretable decomposition of program similarity into operation presence, token position, contiguous-sequence preservation, and edit similarity, applied to two PBE approaches after approach-specific normalization of their structurally different program representations.

This thesis deliberately builds on token-based rather than tree-based comparison, since both approaches can be transformed into a shared normalized token representation without requiring a common tree grammar.

2.3 The Evaluated Approaches: DeepCoder and DreamCoder

To examine whether token-based metrics generalize across approaches, this thesis applies them to two existing PBE approaches that represent and search for programs in entirely different ways. DeepCoder searches a fixed, hand-written DSL using a neural network that predicts which operations a solving program likely contains [1]. DreamCoder instead represents programs as typed lambda-calculus expressions and grows its own library of reusable components over repeated learning cycles [7]. This section describes the program representations and search mechanisms of both approaches.

2.3.1 DeepCoder

DeepCoder instantiates a general framework the authors call Learning Inductive Program Synthesis (LIPS), which factors program synthesis into four components: a DSL together with an attribute function, a procedure for generating training data, a machine learning model, and a search procedure that the model's predictions guide [1].

DSL and Attributes. DeepCoder's DSL is loosely inspired by query languages such as SQL, in which high-level functions are applied in sequence to manipulate data. A program is a sequence of function calls, where each call initializes a fresh variable from the inputs or from previously computed variables, and the program's output is the value of the last variable. DeepCoder predicts 34 DSL function and lambda attributes in total: first-order list functions such as `HEAD`, `LAST`, `TAKE`, `DROP`, `ACCESS`, `MINIMUM`, `MAXIMUM`, `REVERSE`, `SORT`, and `SUM`, higher-order functions such as `MAP`, `FILTER`, `COUNT`, `ZIPWITH`, and `SCANL1`, and the fixed lambda functions supplied as arguments to the higher-order functions (for example `(*2)`, `(>0)`, or `MIN`). LIPS defines an *attribute function* that maps a program to a fixed-length binary vector recording whether each of the 34 function and lambda attributes occurs in it anywhere; this attribute vector is the interface between the neural network and the search procedure, since the network only needs to predict which operations a program likely uses, not their exact arrangement.

Data Generation and Model. Training data is generated by enumerating DSL programs and heuristically pruning those with detectable redundancies, such as a variable that does not affect the output or a shorter behaviorally equivalent program. For

each retained program, input value ranges are derived by propagating bounded output constraints backward through the program, after which inputs are sampled from the resulting valid ranges and the corresponding outputs are computed by running the program. The resulting input-output examples and binary attribute vectors form the supervised training signal. The network itself has an encoder-decoder structure: the encoder represents each input-output example by one-hot encoding its type (single integer or array), embedding every integer into a learned 20-dimensional vector, and passing the concatenated representation through three hidden layers of 256 sigmoid units each; representations of several examples for the same program are then averaged. The decoder multiplies this pooled representation by a learned matrix to produce one logit per DSL function, trained with a cross-entropy loss so the outputs can be read as independent marginal probabilities that each function occurs in the target program.

Search. These predicted probabilities are used to guide a depth-first search (DFS) that tries DSL functions at each search step in order of predicted likelihood, and a *sort and add* enumeration scheme that starts from the most probable functions and progressively admits less probable ones only once the current active set fails to yield a program within the search budget. The same predictions can also guide existing solvers from the programming-languages community, including the SMT-based tool Sketch and the enumerative-with-deduction tool λ^2, by restricting the operations these solvers consider to the current active set. Balog et al. [1] report that this neural guidance yields an order-of-magnitude speedup over unguided search baselines and over a comparable RNN-based approach, allowing DeepCoder to solve problems of a difficulty comparable to the simplest problems on programming-competition websites. The DSL and its production rules, however, never change during this process. DeepCoder's neural network only reorders an otherwise fixed search space.

DeepCoder's vocabulary therefore remains static throughout the entire search and training process: a program space that is fixed from the outset, differing only in the order in which it is searched.

2.3.2 DreamCoder

DreamCoder frames program induction as Bayesian inference and represents programs as expressions of a polymorphically typed lambda calculus rather than sequences over a fixed DSL [7]. Its central object is a *library* L, a growing set of primitives and learned abstractions that together define a prior $\mathrm{P}[\rho \mid L]$ over programs ρ. Learning proceeds in

repeated *wake-sleep* cycles, each consisting of one wake phase and two interleaved sleep phases, that jointly approximate the following three updates for a set of tasks X:

$$\rho_x = \arg\max_{\rho} \mathrm{P}[\rho \mid x, L] \propto \mathrm{P}[x \mid \rho]\,\mathrm{P}[\rho \mid L] \quad \text{(Wake)}$$

$$L = \arg\max_{L} \mathrm{P}[L] \prod_{x \in X} \max_{\rho \text{ a refactoring of } \rho_x} \mathrm{P}[x \mid \rho]\,\mathrm{P}[\rho \mid L] \quad \text{(Sleep: Abstraction)}$$

$$\text{train } Q(\rho \mid x) \approx \mathrm{P}[\rho \mid x, L] \quad \text{(Sleep: Dreaming)}$$

where $\mathrm{P}[L]$ is a description-length prior that favors compact libraries and $\mathrm{P}[x \mid \rho]$ is 1 if program ρ reproduces the input-output examples of task x and 0 otherwise. The wake phase does not search over all programs exhaustively; it is guided by the recognition model $Q(\rho \mid x)$, which prioritizes higher-probability candidates first, which is what makes the search tractable.

Wake Phase. During waking, DreamCoder searches for a program that solves each task in the current batch, guided by a neural *recognition model* $Q(\rho \mid x)$ that enumerates candidate programs in decreasing order of predicted probability until a timeout is reached; to tolerate cases where several candidate programs solve a task, the $k = 5$ solutions with the highest posterior probability are kept for use in the following sleep phases rather than only the single best one.

Sleep: Abstraction (Consolidation). This thesis refers to DreamCoder's abstraction sleep phase as *Consolidation*. Its objective is to grow the library L by refactoring, that is, rewriting, the programs found during waking into semantically equivalent but syntactically different forms, and incorporating whichever resulting subexpressions best compress the solutions found so far into the library as new primitives. Because a program admits an astronomically large number of possible refactorings, up to roughly 10^{14} for a small example given in the original paper, DreamCoder represents the set of refactorings compactly using a version-space-algebra-like data structure rather than enumerating them individually, bounding the number of evaluation steps a refactoring may take to keep this representation tractable. In this way, Consolidation can discover a reusable primitive such as `map` from only a handful of solved list-processing tasks and add it to the library for later reuse.

Sleep: Dreaming (Recognition). The second sleep phase, referred to here as *Recognition*, trains the neural recognition model $Q(\rho \mid x)$ used during waking. Training pairs are drawn in equal parts from *replays*, tasks the system already solved during waking, and *fantasies*, synthetic tasks obtained by sampling a random program from the current library L and executing it. Fantasies are essential for data efficiency, since DreamCoder is typically trained on only 100 to 200 tasks per domain, far too few to train a high-capacity network on their own, whereas the library can generate an effectively unlimited stream of synthetic training data once it has learned a handful of domain-relevant primitives. Training uses a maximum-a-posteriori objective, which additionally breaks ties among semantically equivalent but syntactically distinct programs by concentrating the model's probability mass on a single canonical form.

Effect and Comparison to DeepCoder. Consolidation and Recognition address complementary sources of search cost: higher-level library primitives let a solution be expressed with fewer function calls, reducing how *deep* the search tree needs to go, while the recognition model down-weights unlikely expansions, reducing how *wide* the search needs to be at each step; Ellis et al. [7] note that overall search difficulty scales roughly as breadth to the power of depth, so that either mechanism alone can already yield large gains. The practical effect can be substantial: in the original paper's own list-processing example, a program that sorts a list requires 32 function calls when expressed only in DreamCoder's initial primitives, which the authors estimate would take in excess of 10^{72} years to find by brute-force enumeration, but only 5 function calls once six wake-sleep iterations have enriched the library, solvable in under 10 minutes of search [7]. Here lies the central structural contrast to DeepCoder. DeepCoder's DSL and its 34 functions stay fixed throughout search; only the network's guidance changes. DreamCoder's library, and therefore its very vocabulary of programs, evolves across the run instead.

Because DeepCoder's fixed DSL and DreamCoder's evolving lambda-calculus library share no common vocabulary, their programs cannot be compared token-by-token without further work. Chapter 4 describes the mapping that makes this comparison possible.

This missing shared vocabulary constitutes the central technical obstacle: it must be resolved before the metrics developed in this thesis can be applied to both approaches at all. Before that, however, comes the theoretical framework in which these metrics are embedded: PS, PBE, and the accuracy metric, at whose limits this thesis begins.

3 Theoretical Background

Three concepts underpin everything that follows: PS, its special case PBE, and the accuracy metric used to evaluate both. This chapter first introduces PS and the DSLs it commonly searches over, then narrows this down to PBE, the specific form of PS this thesis is concerned with, before introducing accuracy, the standard metric used to evaluate PS and PBE approaches, and defining a running example task that is reused throughout the following chapters to illustrate the metrics this thesis develops.

3.1 Program Synthesis

Gulwani et al. [10] define PS as the automatic generation of a program from a specification of the desired behavior. Instead of writing the program by hand, a user or developer describes what the program should do, and a synthesis approach searches for a program that satisfies this description. Such a specification can take several forms, for example a formal logical condition, a natural language description, or a set of input-output examples [10, 16]. Table 3.1 illustrates these forms on the same target transformation, squaring an integer.

Table 3.1: The same transformation expressed through three different specification types.

Specification type	Example
Formal condition	$\forall x.\ P(x) = x \cdot x$
Natural language	"Square the input number."
Input-output examples	$2 \to 4,\ 3 \to 9,\ 4 \to 16$

Each specification type comes with a trade-off. A formal condition is unambiguous but can be as much effort to write as the program itself, while a natural language description is easy to write but often underspecified or open to multiple readings [10]. Input-output examples sit between these two extremes. They are simple to provide, and

they still ground the specification in concrete, checkable behavior. This trade-off is a central reason why PBE has become a widely used PS paradigm in practice.

Domain-Specific Languages. The choice of specification also shapes how a synthesis approach searches for a program. Rather than searching over an unrestricted, Turing-complete programming language, PS approaches typically restrict the space of candidate programs to a DSL: a language that fixes which operations, constants, and program structures are permitted for a given class of tasks [9, 2]. This DSL need not stay fixed throughout the entire search, DreamCoder, for instance, continually extends its own DSL with newly learned building blocks [7], nor does it need to exist at all: LLM-based approaches such as Codex forgo a DSL entirely and generate code directly in a general-purpose programming language such as Python [3]. A DSL for the squaring task from Table 3.1 might contain little more than a single multiplication operation; the list-processing domain used later in this thesis needs a richer vocabulary, with operations such as `SORT` to sort a list or `MAP` to apply a function to every element of a list.

This restriction is what makes searching for a correct program tractable in the first place. Consider a DSL with k available operations and a target program of length n, where each of the n positions in the program is filled by one of the k operations. Without any further restrictions, the number of syntactically possible programs of this length grows as k^n. For a DSL with $k = 20$ operations and a program of length $n = 5$, this already amounts to $20^5 = 3{,}200{,}000$ candidate programs. Restricting the DSL to a general-purpose language with, say, $k = 2{,}000$ available operations and identifiers increases this number to $2{,}000^5 = 3.2 \times 10^{16}$, ten billion times larger for the same program length. This is why PS approaches use small, task-tailored DSLs instead of general-purpose languages such as Python or Java: an exhaustive search over the latter quickly becomes computationally infeasible [10]. At the same time, a DSL that is too restrictive limits which transformations can be expressed at all, so, as Cambronero et al. [2] note, choosing a DSL requires a balance between search tractability and expressiveness.

Regardless of how narrowly a DSL ultimately shapes the search space, what the search is looking for is determined by the type of specification used to describe a task, and it is this form that sets PBE apart from the other PS paradigms.

3.2 Programming by Example

PBE is the special case of PS in which the specification consists of a set of input-output examples rather than a formal condition or a natural language description [10, 16]. Formally, Devlin et al. [5] formalize the goal as finding, from a specification comprising a set of pairs $(I_1, O_1), \ldots, (I_n, O_n)$, a program P such that $P(I_j) = O_j$ for every pair j. The doubling-and-sort task from Chapter 1 is one such specification: a pair like ($[3, 1, 2]$, $[2, 4, 6]$) tells a PBE approach nothing about how to double and sort a list in general, only that this particular input must map to this particular output, and it is left to the search to find a program consistent with pairs like it. Because PBE allows users to specify the desired behavior through examples rather than code, it lowers the barrier to automating a task to providing a handful of examples [14, 9].

Underdetermination. A small number of examples rarely pins down a single program: several different programs can satisfy the same examples while behaving differently on inputs that were not shown. Table 3.2 illustrates this with a task that maps a list of numbers to a single number.

Table 3.2: Two candidate programs, "return the maximum" and "return the last element," both satisfy Examples 1 and 2. Only Example 3 disambiguates between them.

	Input	Output
Example 1	$[5, 2, 9]$	9
Example 2	$[1, 4]$	4
Example 3	$[7, 3, 6]$	6

Given only Examples 1 and 2, both "return the maximum element" and "return the last element" are consistent programs, since the maximum happens to be the last element in both cases. Example 3 breaks this coincidence: the maximum of $[7, 3, 6]$ is 7, but the expected output is 6, the last element. Only the second program generalizes correctly to this new input. This is what is meant by underdetermination in PBE: because examples are an incomplete specification, satisfying the given examples is not sufficient to guarantee that a program has captured the intended transformation [10, 26]. In the extreme case, a program need not even look at the input: if the expected outputs of the given examples happen to coincide, a program that always returns the same constant can already satisfy them without encoding any rule about the input at all.

Underdetermination is also why PBE requires more than matching the given examples verbatim. A PBE approach must reason about which of the potentially many programs consistent with the specification is the one the user actually intended, for instance by preferring a simpler or more general program over one that merely memorizes the given examples. Providing more input-output examples per task reduces, but does not eliminate, this ambiguity: Wang et al. [26] show that it is generally impossible to guarantee that a program will behave correctly on every unseen input from a finite set of examples alone. Consequently, evaluating a PBE approach also requires distinguishing between examples used to specify a task and examples used to test whether the synthesized program actually generalizes.

Two Levels of Splitting. In principle, evaluation in PBE can distinguish between two different levels of splitting. At the level of an individual task, input-output examples may be divided into a specification part, which is given to the synthesis approach, and a held-out part, which can be used to test whether the synthesized program generalizes beyond the examples it saw during synthesis. At the level of an entire dataset, the available tasks can instead be split into a training portion, used to develop or train the approach, and a held-out test portion containing tasks the approach has not encountered before.

The concrete split used in this thesis is specified in Section 5.1.

3.3 Accuracy as an Evaluation Metric

Accuracy is the standard metric for evaluating classification-style predictions and is defined generally as the proportion of correctly predicted examples relative to the total number of examples [20]. This general definition is commonly instantiated for PS and PBE by treating the exact match between a generated and an expected output as the correctness criterion: a task counts as solved only if the generated output matches the expected output on every one of the task's test examples, not merely on some of them [3, 16]. Over a set of tasks, accuracy is then the proportion of tasks solved in this sense:

$$\text{accuracy} = \frac{\text{number of correctly solved tasks}}{\text{number of all tasks}}.$$

A value of 1 means every task was solved, and a value of 0 means none was. Accuracy owes its popularity to being easy to understand, cheap to compute, and directly

comparable across approaches, particularly in benchmarks where each task has a single, unambiguous expected output [20].

At the task level, this complete-match criterion is binary: a task either satisfies the criterion or it does not.

A Running Example Task. To make the following chapters concrete, this thesis returns to the doubling-and-sort task from Chapter 1 and uses it as one running example: transforming a list of integers by doubling each element and sorting the result. Table 3.3 formalizes this task through three input-output examples, together with a reference program that solves it.

Table 3.3: The running example task: doubling and sorting a list of integers.

Input	Output
$[3, 1, 2]$	$[2, 4, 6]$
$[5, -1, 0]$	$[-2, 0, 10]$
$[4, -3, 1]$	$[-6, 2, 8]$

Reference program: Map($\times 2$)(Sort(input))

Under the definition given above, this reference program achieves an accuracy of 1 on the task in Table 3.3, since it reproduces the expected output on every example. Now consider a second program, Map($+2$)(Sort(input)), which sorts the input correctly but adds 2 to each element instead of doubling it. This program produces $[3, 4, 5]$ instead of $[2, 4, 6]$ for the first example, so it fails on every test example and receives an accuracy of 0, the same score as a program with no discernible relation to the task at all. Yet the two programs are not equally wrong. The second one correctly identifies that the input must be sorted and applies an operation to each element; it deviates from the reference program in only one respect. Accuracy, being binary by construction, cannot express this difference. A program that noticeably comes closer to the reference than an arbitrary wrong one receives the same value of 0 as that program.

A program that comes close to the reference, however, clearly deserves more than a zero. This calls for a yardstick that does not stop at the output but looks at the program itself: its structure, its building blocks, and their arrangement.

4 Methodology

Answering the research question requires a concrete methodology, developed here in three parts. This chapter resolves a problem raised at the end of Chapter 2: DeepCoder's fixed DSL and DreamCoder's evolving lambda-calculus library share no common vocabulary, so their programs must be brought onto comparable ground before anything about them can be compared. Building on this shared vocabulary, four token-based metrics quantify how closely a generated program matches a reference; the chapter closes by explaining how these metrics are read jointly, together with the BSS, to answer the research question.

4.1 Data Conversion

DeepCoder's programs are already sequences of DSL operation calls, so tokenizing them is direct: each operation, together with its arguments, becomes one token. DreamCoder's programs, by contrast, are expressions of a typed lambda calculus and can express the same operation in many different but semantically equivalent syntactic forms. A direct, token-by-token comparison between a DeepCoder reference program and a DreamCoder solution would therefore be meaningless even when both compute the same transformation. Identical behavior simply would not translate into identical tokens.

To close this gap, DreamCoder's lambda-calculus expressions are mapped onto a shared normalized operation vocabulary centered on the DeepCoder operation families, independently of how a given operation happens to be nested or indexed within the expression. A lambda-calculus subexpression that reverses a list, for instance, is mapped to the same token that DeepCoder's DSL uses for its built-in reverse operation, regardless of the exact variable binding used to express it. This mapping operates at the level of operation *families* rather than exact syntax: it recognizes that two expressions perform the same kind of operation even when one spells it out more explicitly than the other. The normalized vocabulary is centered on these DeepCoder operation families but is not restricted to the 34 native DeepCoder primitives; normalization-specific ab-

stract tokens are retained where no direct DeepCoder primitive provides an appropriate correspondence. A representative case from the evaluation illustrates this: a reference program consisting of a `TAKE` and a `REVERSE` operation is matched against a DreamCoder solution that reaches the same result differently, by first concatenating the input onto an empty list and only then reversing the concatenated result, a functionally equivalent but syntactically longer detour through three operation tokens instead of one. After normalization, both programs are represented in a shared operation vocabulary and can be compared using the token-based metrics defined next, even though their sequence lengths, syntactic forms, and level of detail may differ.

Operations for which no correspondence is defined remain unmapped; despite this, most program pairs share a common token vocabulary after normalization.

4.2 Token-based Program Metrics

This thesis introduces four such metrics, each isolating a different way a generated program can deviate from the reference: the Program Operation Score (POS), the Program Position Score (PPS), the Program Sequence Score (PSS), and the Program Edit Score (PES). A single aggregate similarity score, such as BLEU (Section 2.2), could not distinguish an approach that finds the right operations but combines them in the wrong order from one that fails to find the right operations at all, even though both could produce the same aggregate value; keeping presence, position, contiguous structure, and overall edit effort as four separate numbers instead makes it possible to attribute a low score to a specific kind of structural deviation. By convention, if both the reference and the generated token sequence are empty, every metric below evaluates to 0.

To illustrate the four metrics with the same token representation used in the evaluation, two valid four-operation DeepCoder programs are paired below. The pair serves only as an illustration; the evaluated reference programs themselves contain two operations. A four-operation pair is used only to make the different structural signals visible in one compact example; the metric definitions themselves do not depend on this program length. Both programs have the signature

$$p, \hat{p} : \mathrm{List}(\mathbb{Z}) \to \mathrm{List}(\mathbb{Z}).$$

Let x_0 denote the input list. The reference program is

```
LIST|COUNT,<0,0|COUNT,>0,0|DROP,1,0|TAKE,2,3.
```

Written in terms of the intermediate variables referenced by the DSL calls, the reference program is:

$$\begin{aligned} &x_0 : \mathrm{List}(\mathbb{Z}) && \text{input},\\ &x_1 : \mathbb{Z} = \mathrm{COUNT}(< 0, x_0),\\ &x_2 : \mathbb{Z} = \mathrm{COUNT}(> 0, x_0),\\ &x_3 : \mathrm{List}(\mathbb{Z}) = \mathrm{DROP}(x_1, x_0),\\ &x_4 : \mathrm{List}(\mathbb{Z}) = \mathrm{TAKE}(x_2, x_3). \end{aligned}$$

The comparison program

```
LIST|COUNT,<0,0|DROP,1,0|COUNT,>0,0|TAKE,3,2
```

uses the same computations but assigns the intermediate variables in a different order:

$$\begin{aligned} &x_0 : \mathrm{List}(\mathbb{Z}) && \text{input},\\ &x_1 : \mathbb{Z} = \mathrm{COUNT}(< 0, x_0),\\ &x_2 : \mathrm{List}(\mathbb{Z}) = \mathrm{DROP}(x_1, x_0),\\ &x_3 : \mathbb{Z} = \mathrm{COUNT}(> 0, x_0),\\ &x_4 : \mathrm{List}(\mathbb{Z}) = \mathrm{TAKE}(x_3, x_2). \end{aligned}$$

The numeric arguments in the DSL representation refer to the input or to variables produced by earlier operations. Thus, for example, `DROP,1,0` applies the count stored in x_1 to the input list x_0, whereas `TAKE,2,3` uses the integer stored in x_2 and the list stored in x_3. Although both programs compute the same transformation, their operation sequences differ structurally. Table 4.1 confirms this on two example inputs.

Table 4.1: Two short executions showing that both programs compute the same transformation.

Input	$p(x)$	$\hat{p}(x)$
$[-247, 162]$	$[162]$	$[162]$
$[-28, 69, 102]$	$[69, 102]$	$[69, 102]$

The leading `LIST` marker specifies the input type and is not part of the scored sequence. Each executable DSL call together with its arguments forms one composite token. The

two scored sequences are therefore

$$p = [\texttt{COUNT,<0,0}, \texttt{COUNT,>0,0}, \texttt{DROP,1,0}, \texttt{TAKE,2,3}]$$

and

$$\hat{p} = [\texttt{COUNT,<0,0}, \texttt{DROP,1,0}, \texttt{COUNT,>0,0}, \texttt{TAKE,3,2}].$$

4.2.1 Program Operation Score

The POS asks a narrow question first: which reference building blocks are present in the candidate, regardless of where they occur? It treats the two token sequences as multisets instead of ordered sequences, so neither order nor position affects the result. A plain set-based check would already be satisfied by a single occurrence of a required operation; since PBE programs frequently reuse the same operation with different arguments, for instance in chained `Map` calls, a multiset comparison is used instead, so that a reference token must occur the same number of times in the candidate to receive full overlap credit, not merely once.

$$\mathrm{POS}(p, \hat{p}) = \frac{\sum_{t \in T} \min(\mathrm{count}_p(t), \mathrm{count}_{\hat{p}}(t))}{\max(|p|, |\hat{p}|)}$$

Here T is the set of all tokens occurring in p or $\hat{p}$, and $\mathrm{count}_p(t)$ counts how often token t occurs in p. Three of the four reference tokens also occur in $\hat{p}$: `COUNT,<0,0`, `COUNT,>0,0`, and `DROP,1,0`. The final tokens differ because the changed execution order also changes the variable references from `TAKE,2,3` to `TAKE,3,2`. The POS is therefore $3/4 = 0.75$. Their positions do not affect this value: a candidate with completely swapped operations can therefore receive the same POS as one that places the shared tokens at their reference positions.

4.2.2 Program Position Score

Index by index, the PPS compares both sequences without any alignment step: it does not first align the two sequences, the way the PES does below via edit-distance alignment, and only then measure how far tokens drifted. An alignment-tolerant variant would absorb exactly the position sensitivity this metric is meant to expose, making it largely redundant with the PES; the raw, unaligned comparison instead penalizes any positional shift, however small, so that ordering differences stay visible.

$$\mathrm{PPS}(p, \hat{p}) = \frac{\sum_{i=1}^{\min(|p|,|\hat{p}|)} \mathbf{1}(p_i = \hat{p}_i)}{\max(|p|, |\hat{p}|)}$$

where $\mathbf{1}(p_i = \hat{p}_i)$ is 1 if the tokens at position i match and 0 otherwise. Only the first position matches exactly, as Table 4.2 shows:

Table 4.2: Position-by-position comparison between p and $\hat{p}$.

Position	p	$\hat{p}$	Match
1	`COUNT,<0,0`	`COUNT,<0,0`	yes
2	`COUNT,>0,0`	`DROP,1,0`	no
3	`DROP,1,0`	`COUNT,>0,0`	no
4	`TAKE,2,3`	`TAKE,3,2`	no

The positive-count and drop operations exchange positions, and the final `TAKE` token also differs because its variable references change. Hence $\mathrm{PPS}(p, \hat{p}) = 1/4 = 0.25$, despite three reference tokens being present somewhere in the candidate. What the PPS makes visible here is exactly the reordering that the POS, by design, cannot see. Whether the shifted tokens at least remain together as a contiguous block, however, is not something position alone can determine, since it penalizes every shift independently, regardless of whether neighboring tokens move together or are scattered individually across the sequence.

4.2.3 Program Sequence Score

This thesis uses a strict variant of the LCS-based comparison, measuring the longest *contiguous* common token block rather than the classical, gapped LCS. A gapped LCS would reward relative-order preservation even when matching tokens are separated by unrelated tokens. This thesis deliberately uses a contiguous variant instead, because the aim is to isolate adjacency preservation as a stricter structural property: operations that sit directly next to each other in the reference program are rewarded only if they also remain adjacent in the generated program.

$$\mathrm{PSS}(p, \hat{p}) = \frac{\mathrm{LCB}(p, \hat{p})}{\max(|p|, |\hat{p}|)}$$

where $\mathrm{LCB}(p, \hat{p})$ is the length of the longest contiguous block common to both sequences. No common contiguous block of length two survives the reordering: although

the first three reference tokens all occur in $\hat{p}$, the longest contiguous block shared by both sequences contains only one token. $\mathrm{PSS}(p, \hat{p})$ works out to $1/4 = 0.25$ as a result. POS, PPS, and PSS thus provide three separate views of presence, position, and contiguity, but they do not quantify the total number of edits required to match the reference; this is captured by the PES.

4.2.4 Program Edit Score

Overall edit effort is measured as the number of changes required to transform one token sequence into the other. Plain Levenshtein distance, allowing insertion, deletion, and substitution, is used rather than a variant such as Damerau-Levenshtein that additionally allows transposing adjacent tokens: a transposition is already reflected in the PPS and PSS above, so adding it here would make the PES partly redundant with those two metrics instead of contributing an independent, aggregate signal.

$$\mathrm{PES}(p, \hat{p}) = 1 - \frac{\mathrm{EditDistance}(p, \hat{p})}{\max(|p|, |\hat{p}|)}$$

A value of 1 means the two programs are token-for-token identical; lower values mean more edits are required to turn one into the other. Transforming $\hat{p}$ into p requires three edits under ordinary Levenshtein distance: the first token already matches, and substituting the tokens at positions two, three, and four yields the reference sequence. That puts $\mathrm{EditDistance}(p, \hat{p})$ at 3, so $\mathrm{PES}(p, \hat{p}) = 1 - 3/4 = 0.25$. That this value coincides with the PPS and PSS for this particular pair of programs is not a general pattern; it reflects that these three substitutions happen to cost as many edits here as they cost matched positions and contiguous blocks. The POS, at 0.75, remains the outlier of the four: it is the only one of the four metrics blind to the reordering itself, which is precisely what lets it show that three reference tokens are nevertheless shared by the two programs.

With POS, PPS, PSS, and PES, four numbers are therefore available for each pair of programs, each expressing a different kind of structural deviation, from pure presence to aggregated edit effort. Four separate numbers are not yet an interpretation, however: what a high POS combined with a low PPS actually says about a candidate, and how these four values relate to the BSS that alone decides whether a task is solved or not, only becomes clear once they are read together.

4.3 Comparison and Interpretation of the Metrics

Before the rest of this section turns to how the four metrics are read together, Table 4.3 summarizes the structural property captured by each metric and its value on the illustrative DeepCoder pair.

Table 4.3: The four token-based metrics and their values on the illustrative DeepCoder pair.

Metric	What it captures	Illustrative example
POS	Presence of reference tokens (multiset), ignoring order and position	0.75
PPS	Whether tokens occur at exactly the same position	0.25
PSS	Length of the longest contiguous common token block	0.25
PES	Overall number of token edits needed to match the reference	0.25

The four metrics are not intended to be read in isolation, but jointly, since each captures a different way a generated program can diverge from the reference. A high POS together with a low PPS indicates that many reference tokens are preserved in the generated program but do not occur at their reference positions. A low POS, by contrast, means components of the reference program are missing or were replaced outright. Contiguous parts of the reference structure survive as a unit when the PSS is high; when it is low, at best isolated tokens match without forming any shared substructure. The PES complements this by quantifying the total edit effort behind whatever pattern the other three metrics reveal.

The BSS remains informative at the output level: it shows whether the final output is correct. The four program-level metrics add a complementary layer by showing why an output might be wrong, or why it might be right for reasons the BSS cannot distinguish. Because several different programs can produce the same output, a generated program can be judged correct at the output level while still deviating structurally from the reference; in such a case, the BSS reaches its maximum value while the token-based metrics can still be comparatively low: the correct output, it turns out, was produced by a structurally different program. Conversely, a generated program can be judged entirely wrong at the output level while still scoring highly on individual token-based metrics. Substantial parts of the reference structure were correctly recognized here, even though the final output was not. Reading the BSS together with POS, PPS, PSS, and PES therefore enables a more differentiated interpretation of a result than either could provide alone.

The four metrics measure structural similarity to a reference program rather than semantic equivalence. Their joint interpretation therefore shows whether structural deviations coincide with correct or incorrect outputs on the evaluated examples.

5 Evaluation

Before any result can be reported, four things have to be fixed: the task domain both approaches are evaluated on, the specific configurations compared within each approach, the hyperparameters and reproducibility guarantees behind every number, and the convention used to aggregate and report them. This chapter fixes each in turn, in that order.

5.1 Task Domain

The list processing domain concerns the transformation of integer arrays, using the 34 DeepCoder primitives introduced in Section 2.3.1, which define the operation vocabulary from which DeepCoder programs in this evaluation are constructed.

A program is written as a sequence of these operation calls, each one creating a new variable from the task's inputs or from variables computed by earlier calls; the value of the last variable is the program's output. Each program string is additionally prefixed by one type marker per input parameter, `LIST` or `INT`, stating that parameter's type; since any candidate considered for a given task necessarily shares the reference's input types, this prefix carries no information about how close a candidate's computation actually is and is therefore excluded before the four token-based metrics defined in Chapter 4 are computed, leaving one composite token per executable DSL operation call. Program length counts these operation calls rather than raw tokens: a lambda function paired with a higher-order operation, such as multiplying every element by -1 inside a `MAP` call, is counted as a single operation together with that call, not as two separate steps. All reference programs in the evaluated dataset contain exactly two chained DeepCoder operations. DeepCoder candidates are evaluated under the corresponding two-operation task setting, whereas generated DreamCoder solutions are not restricted to two normalized operation tokens and may map to shorter or longer sequences.

Table 5.1 illustrates this on a real task from the underlying dataset. Its reference

program first negates every element of the input list, then returns the last element of the result, equivalently and more simply, it returns the negation of the input's last element.

Table 5.1: A list processing task with five input-output examples, together with a reference program of length two that satisfies all of them.

	Input	Output
Example 1	$[-205, -137, 172, 94, 179, 143, 144, -15, 80, 195, 196, 146]$	-146
Example 2	$[149, 100, -247, -215, 114]$	-114
Example 3	$[-193, 77, 212, -206, 134, 113, -66, 251, -85, -229, 157, -14]$	14
Example 4	$[120, -119, -247, 236, -102, 210, 196, 248, 84, 172, 225]$	-225
Example 5	$[-74, -229, 182, -199, 37, 210, 166, -151, -89]$	89

Reference program: `LIST|MAP,*-1,0|TAIL,1`

Task inputs vary between one and three such arguments, each either a single integer or a list of up to 20 integers, with all integer values, list elements included, ranging between -256 and 256. Input values are not sampled uniformly at random; the permissible range for each argument is first propagated backward from the operations the reference program applies to it, so that every sampled input keeps the resulting computation well-defined and within this range. The corresponding output is then obtained simply by running the reference program on the sampled input. Each task is specified by five such input-output examples, and a task counts as solved once a search finds a program consistent with all five within the available search budget. For the length restriction used in this thesis, 100 test tasks and 2,365 training tasks are available; the training portion is used to fit each approach's learned component, and all reported metrics are computed on the held-out test portion only.

For DreamCoder, programs are normalized as described in Section 4.1 before the four token-based metrics are applied. Of the original 100 test tasks, one is excluded during conversion to the DreamCoder task format because it contains no usable input-output examples, leaving 99 test tasks for evaluation; all three DreamCoder configurations examined in this thesis are trained on a 500-task subset of the corresponding training split.

Within this shared domain, however, DeepCoder and DreamCoder do not compete against each other as single, fixed approaches: each is run in several configurations that specifically isolate individual mechanisms, in order to clarify which part of an approach is actually responsible for a result.

5.2 Compared Configurations

The base code for both approaches builds on the publicly available implementations of DeepCoder [12] and DreamCoder [8]. The extensions required for this thesis, the mapping of DreamCoder's expressions onto the shared normalized operation vocabulary, the four token-based metrics this thesis contributes, and a mechanism for tracking partial candidates on unsolved tasks, were implemented on top of these bases and are published at [28] and [29], respectively.

For DeepCoder, the search itself is evaluated in two configurations that isolate the contribution of its neural component: a plain depth-first search over the 34 primitives, tried in a fixed default order, and the same search guided instead by a neural attribute predictor after it has been trained on the training split to estimate which of the 34 primitives a solving program is likely to contain. Comparing these two configurations at the same fixed search budget separates what an unguided search can already find on its own from what the neural guidance adds on top of it.

For DreamCoder, its two learning mechanisms are evaluated both individually and combined, giving three configurations per run: Consolidation without Recognition, Recognition without Consolidation, and both mechanisms together. All three configurations are run for the same fixed number of training cycles on the same training tasks, allowing their observed differences to be examined under otherwise identical training conditions.

This settles five named configurations, but their names alone do not yet determine whether a measured difference between them actually reflects the respective mechanism or merely random fluctuation: for that, the search budget, training duration, and handling of randomness behind every individual number must also be fixed.

5.3 Hyperparameters and Reproducibility

Unless stated otherwise, the DeepCoder results refer to a search budget of $gas = 1{,}000$ candidate programs per task. The DreamCoder configurations are each trained for 3 wake-sleep cycles with an abstraction arity of 3, meaning that a newly learned library component may combine up to three existing pieces at once.

For DeepCoder, the plain, untrained configuration is fully deterministic: its depth-first search contains no learned or randomized component, so repeated runs are byte-identical regardless of seed. The trained configuration depends on the attribute predictor's weight

initialization and training-batch order, both seed-dependent; it is trained for 10 epochs and reported individually for each of five seeds (0–4), alongside the mean and population standard deviation across them, both in the result tables and in the accompanying bar charts. DreamCoder exposes a random-seed parameter as well, but it only affects a task-batching strategy that this thesis does not use and has no effect on library learning or the recognition model under the configuration used here; each DreamCoder configuration is therefore reported as a single run rather than an aggregate over several. DreamCoder's enumeration and testing timeout are each set to 15 seconds per task.

Five seed-dependent DeepCoder runs and a single DreamCoder run per configuration, however, do not by themselves yield a single reportable number: in particular for tasks on which a search fails to find a solving program, it must first be settled whether and how they factor into a mean at all.

5.4 Reporting Convention

For every metric, both the mean over *all* test tasks and the mean over only the *solved* tasks are reported. Programs that satisfy all five given examples are counted as solved by the BSS even when they are later identified as trivial input-ignoring solutions. Such cases therefore remain included in the solve-rate counts and solved-only structural averages, but are flagged separately for qualitative analysis. Unsolved tasks enter the token-based metrics with the value 0 by convention, regardless of whether an incomplete candidate was captured during the search; reporting only the all-task mean would therefore understate how structurally close the search actually gets on the tasks it does solve, regardless of how many tasks remain unsolved overall.

For DeepCoder, an additional best-effort candidate is tracked during search for unsolved tasks. Among all valid partial programs visited within the search budget, the candidate satisfying the largest number of the five given input-output examples is retained. Candidates that raise a `NullInputError` or `OutputOutOfRangeError` are discarded. If several candidates satisfy the same maximum number of examples, the first one encountered in the search order is retained. Consequently, the selected candidate depends on the actual search order and the fixed gas budget. If no valid visited candidate satisfies even one example, no best-effort candidate is stored. The reference program and the four structural metrics are not used during this selection; they are applied only afterwards for evaluation.

Domain, configurations, hyperparameters, and aggregation convention together thus

determine which number may be reported at all for which run.

6 Results and Discussion

This chapter examines three expectations derived from the research question. First, a BSS hides aspects of search behavior that become visible at the program level. Second, partial correctness can provide additional information even for tasks that remain unsolved under the BSS. Third, program-level metrics can provide a more informative account of why a task was or was not solved than the BSS alone. Performance and structural similarity are first reported separately and then related for solved and unsolved tasks. The analysis concludes by considering the limits of what these measurements can establish.

6.1 Performance

Table 6.1 reports the solve rate, the BSS averaged over a set of tasks, for each configuration examined.

Table 6.1: Solve rate (BSS) across all five configurations.

Configuration	Tasks	Solved	Solve rate
DeepCoder, without training	100	53	53.0%
DeepCoder, with training, mean ± std	100	67.8	67.8% ± 1.8%
DreamCoder, Consolidation	99	25	25.3%
DreamCoder, Recognition	99	31	31.3%
DreamCoder, combined	99	34	34.3%

For DeepCoder, training the attribute predictor raises the solve rate substantially at the same search budget, from 53.0% to a mean of 67.8% ± 1.8% across five seeds, with individual seeds ranging from 66 to 70 solved tasks out of 100. For DreamCoder, the combined configuration reaches the highest solve rate at 34.3%, ahead of Recognition alone (31.3%) and Consolidation alone (25.3%). Since all three configurations use the same training tasks and iteration count, the observed differences cannot be explained by

these experimental conditions alone. The higher solve rate of the combined configuration is consistent with complementary effects of Consolidation and Recognition under the tested setting. The aggregate counts alone, however, do not establish how the sets of solved tasks overlap or which tasks are unique to each configuration.

The solve rate alone, however, says nothing about how structurally close a generated program is to its reference.

6.2 Structural Similarity of the Generated Programs

Even among solved tasks, not every generated program is the reference program itself. The fraction of solved tasks whose program is token-for-token identical to the reference, equivalently the fraction reaching a PES of 1, is comparatively small: for DeepCoder, only 11.0% of all tasks (20.8% of the solved ones) are matched exactly without training, rising to a mean of 21.0% $\pm$ 1.3% of all tasks (30.9% $\pm$ 1.1% of the solved ones) with training. For DreamCoder, exact matches are rarer still: neither Consolidation nor the combined configuration produces an exact match, while Recognition produces one, corresponding to 1.0% of all test tasks and 3.2% of its solved tasks. In the large majority of solved cases, the search therefore finds an alternative program instead, one that satisfies all five given examples but is structurally different from the reference, precisely the distinction accuracy alone cannot draw.

What the two training procedures reward may help explain why many solved tasks nonetheless fall short of a PES of 1. DeepCoder's attribute function maps a program onto a binary vector that only records whether a function occurs anywhere, not in what order or nesting; the network consequently learns to predict operations, never an arrangement. The subsequent depth-first search does try these predicted functions in descending order of probability, but among the example-satisfying compositions it finds, it does not favor one that resembles any particular reference structure, since no such reference is ever known to the training process. Every candidate program that reproduces the same examples counts equally for training. This is consistent with the observation that solved programs can contain the right building blocks without arranging them in the reference's order: the training objective itself does not distinguish between one valid arrangement and another as long as both produce the same output.

For DreamCoder, the same effect sits even deeper inside the training loop itself. Consolidation extends the library according to a prior $P[L]$ that explicitly favors compact libraries, and adopts as new primitives precisely those sub-expressions that most strongly

compress the corpus of tasks solved so far, regardless of how closely the resulting solution matches the decomposition used by any one reference program. Recognition, in turn, concentrates its probability mass on a single canonical form whenever several syntactically different but semantically equivalent programs solve a task, but that canonical form is whichever one the current library itself makes most likely, not the external reference from the evaluation dataset, which the model never sees during training, since its training pairs come exclusively from its own solved tasks and self-sampled fantasies. A training objective that favors compact representations across tasks has no explicit incentive to move toward a task-specific reference structure and may instead favor a more compact, cross-task-reusable alternative when one exists.

For the metrics developed in this thesis, this means they consistently measure something whose optimization neither training procedure ever rewards. A PES below 1on a solved task is therefore consistent with a training objective that rewards example correctness but assigns no value to structural closeness to a specific reference, rather than being evidence that the underlying search worked incorrectly. Since no optimization pressure pushes in this direction, the four token-based metrics supply information here that neither the BSS nor the training signal itself already contains. They are reported individually below, beginning with the least strict of the four.

6.2.1 Program Operation Score

Table 6.2: POS, averaged over all tasks and over only the solved tasks.

Configuration	All tasks	Solved only
DeepCoder, without training	0.180	0.340
DeepCoder, with training, mean ± std	0.274 ± 0.014	0.404 ± 0.011
DreamCoder, Consolidation	0.064	0.253
DreamCoder, Recognition	0.085	0.272
DreamCoder, combined	0.079	0.230

DeepCoder's POS (Table 6.2) is higher than DreamCoder's in every configuration. Part of this difference can be attributed to the different program representations and normalization procedures. DreamCoder solutions may normalize to sequences that are shorter or longer than the two-operation reference programs. Longer generated sequences increase the denominator of the POS and can therefore reduce the score even when

relevant reference operations are present. At the same time, mapping syntactically different DreamCoder expressions onto broader operation families can increase overlap by collapsing representational differences. The resulting POS difference can therefore not be attributed to token granularity in a single direction and should not be interpreted as a representation-independent ranking of the two approaches' search performance.

The POS treats both token sequences as multisets and completely ignores where a token sits; a candidate with fully swapped operations would score the same as one that places every token correctly.

6.2.2 Program Position Score

Table 6.3: PPS, averaged over all tasks and over only the solved tasks.

Configuration	All tasks	Solved only
DeepCoder, without training	0.145	0.274
DeepCoder, with training, mean ± std	0.268 ± 0.017	0.395 ± 0.016
DreamCoder, Consolidation	0.028	0.110
DreamCoder, Recognition	0.026	0.083
DreamCoder, combined	0.049	0.142

Every configuration scores lower on the PPS (Table 6.3) than on the POS reported above, confirming that the tokens found are not always where the reference program places them. The drop is much steeper for DreamCoder: its PPS never exceeds 0.142, far below its own POS, while DeepCoder's PPS stays close to its POS. This asymmetry is not simply a consequence of DreamCoder's library changing over the course of a run; even with a fixed library, there would be no reason why a token's position in a normalized lambda-calculus expression should match its position in a sequentially composed DeepCoder DSL program. DeepCoder's DSL forces an operation into a specific place in the call sequence, fixed by the data dependencies between steps; DreamCoder's lambda-calculus expressions have no analogous sequential structure, and their position after normalization is an artifact of how the expression tree is serialized, not of the functional order of the computed steps. This difference in representation may contribute to DeepCoder's PPS remaining close to its POS in the observed runs. Once DeepCoder finds the right operations, the data dependencies between steps can constrain the number of valid arrangements. The evolving DreamCoder library may further contribute to

this effect, since the same operation can be represented through different active library components at different stages of a run.

The PSS provides the complementary view by measuring how much contiguous structure is preserved between the two programs.

6.2.3 Program Sequence Score

Table 6.4: PSS, averaged over all tasks and over only the solved tasks.

Configuration	All tasks	Solved only
DeepCoder, without training	0.180	0.340
DeepCoder, with training, mean ± std	0.274 ± 0.014	0.404 ± 0.011
DreamCoder, Consolidation	0.054	0.213
DreamCoder, Recognition	0.072	0.228
DreamCoder, combined	0.071	0.206

The PSS (Table 6.4) stays clearly below the POS for DreamCoder, but coincides with it exactly for DeepCoder in every configuration examined. This equality is an empirical property of the evaluated DeepCoder program pairs rather than a consequence of the two-operation restriction alone. In the observed pairs, operation overlap takes only three forms: no shared operation, one shared operation, or both operations shared in the same order. Under these observed patterns, every operation overlap counted by the POS also forms a contiguous block counted by the PSS, causing both metrics to obtain the same value. The two-operation setting restricts the possible overlap patterns, but does not by itself guarantee equality between POS and PSS.

Relative to the PPS, both approaches now agree in kind, if not in degree: the PSS exceeds the PPS in every configuration for both. For DeepCoder the margin is small, between 2% and 24% higher among solved tasks depending on configuration. For DreamCoder, by contrast, the PSS is far higher than the PPS in every configuration, between 45% and 175% higher among solved tasks. Since the PSS only rewards tokens staying adjacent, while the PPS penalizes even the smallest shift independently, this much larger gap shows that DreamCoder's solutions more often share a contiguous sub-block with the reference that is merely shifted to a different position, rather than being fully scattered, a pattern the PPS alone cannot make visible.

Relative to the POS, the PSS is also considerably more stable than the PPS for Dream-

Coder. For DreamCoder, the PSS reaches 84 to 90% of its own POS among solved tasks throughout: 0.213 out of 0.253 for Consolidation, 0.228 out of 0.272 for Recognition, 0.206 out of 0.230 for combined. The PPS's share of the POS, by contrast, fluctuates for DreamCoder only between 31% and 62%, markedly lower and far less consistent across the three configurations. Even when the search finds the right operations, then, how many of them land exactly in the right position remains uncertain for DreamCoder, yet a large, largely stable share of them at least form a contiguous block with the reference somewhere. This is because tree serialization tends to keep semantically related subexpressions together as a block, for instance an operation together with the operation supplying its argument, even as where in the overall sequence that block ends up shifts; a growing library or a different serialization order therefore tends to shift the block as a whole rather than tearing it apart internally. The PSS is accordingly the more robust of the two indicators against the representation-related positional shifts that may contribute to the substantially lower PPS values observed for DreamCoder.

POS, PPS, and PSS thus each illuminate a separate facet of program similarity for DreamCoder; for DeepCoder at this program length, POS and PSS collapse onto the same value, while the PPS remains a genuinely distinct, stricter view. None of the three alone sums up how much total editing lies between the generated and the expected program; the last of the four metrics provides that overall view.

6.2.4 Program Edit Score

Table 6.5: PES, averaged over all tasks and over only the solved tasks.

Configuration	All tasks	Solved only
DeepCoder, without training	0.145	0.274
DeepCoder, with training, mean ± std	0.268 ± 0.017	0.395 ± 0.016
DreamCoder, Consolidation	0.033	0.130
DreamCoder, Recognition	0.045	0.142
DreamCoder, combined	0.064	0.186

For DreamCoder, the PES (Table 6.5) lies between the PPS and PSS in every configuration, not close to either one: an edit sequence can compensate for a shifted but intact block through a few insertions and deletions, and so credits part of the pattern the PSS fully captures and the PPS completely misses, though not to the same degree

as the PSS itself. For DeepCoder specifically, the PES equals the PPS exactly in every configuration in this table. This equality is not a coincidence of this particular dataset but a direct consequence of the two-operation length restriction: for two token sequences of length two, using any insertion or deletion at all requires a matching counterpart to keep the length unchanged, at a combined cost of at least two, no cheaper than simply substituting both mismatched tokens directly; the cheapest edit sequence between two length-two programs is therefore always a straight run of position-wise substitutions, the same operation the PPS penalizes. The number of edits needed and the number of positions that already differ are consequently always the same number at this program length, which is why PES and PPS collapse into a single value for DeepCoder. This equality is specific to programs of exactly this length and should not be expected to hold for longer ones, one reason this thesis keeps the PPS and PES as separate metrics rather than assuming one could stand in for the other.

As an aggregate measure, the PES can also be read directly as editing effort. For DeepCoder's trained configuration, a PES of 0.395 among solved tasks means that, on average, roughly three fifths of the tokens would already need to change to get from the generated to the reference program, even among the tasks counted as fully solved. For DreamCoder's combined configuration, this share is still higher, exceeding four-fifths (a PES of 0.186): a task solved under the BSS is thus, on average, farther from its reference than a pure presence or continuity view alone would suggest, for both approaches, though markedly more so for DreamCoder.

Within the three DreamCoder configurations, the PES also shows a pattern opposite to the PSS's: while the combined configuration reaches the lowest PSS of the three among solved tasks, it reaches the highest PES, 0.186 against 0.142 for Recognition and 0.130 for Consolidation. This traces back to its markedly higher PPS, 0.142 against 0.083 and 0.110. This higher PPS value, however, can no longer be attributed to the search alone: since each of the three configurations actually solves a different subset of the 99 tasks, a solved-only mean compared across the three configurations does not compare the same set of tasks, only whichever tasks each configuration manages to solve at all; how much of combined's higher PPS reflects the additionally solved tasks themselves and how much reflects genuinely more consistent positioning cannot be separated with the data available here. Because the PES lies between the PPS and PSS, this unusually high positional accuracy nonetheless pulls the aggregate value upward, even though continuity is weaker than in the other two configurations.

Reporting all four metrics separately makes it possible to compare their relative be-

havior with the observed solve rates.

6.3 Connection between Performance and Structural Similarity

A first hint that performance and structural similarity can diverge comes from DreamCoder's combined configuration. Although it reaches the highest solve rate among the three DreamCoder configurations (Table 6.1), its PSS among solved tasks (0.206) is the lowest of the three, not the highest, a first, concrete instance of the disagreement between the BSS and the structural metrics that this thesis set out to make visible. A task-level comparison with Recognition provides a more direct explanation of this difference. The combined configuration solves five tasks that Recognition does not solve, and all five obtain a POS, PPS, PSS, and PES of exactly 0. Recognition, conversely, solves two tasks that the combined configuration does not, resulting in a net increase of three solved tasks from 31 to 34. The higher solve rate of the combined configuration therefore coincides with the inclusion of several solutions that satisfy all five given examples while showing no measured structural overlap with their reference programs. This illustrates how a higher solve rate can coexist with a lower solved-only PSS without establishing which internal DreamCoder mechanism caused these particular solutions to be found. The comparison across all configurations tests whether this divergence extends beyond the combined DreamCoder configuration.

6.3.1 Magnitude of the Gap between POS and the Other Structural Metrics

One observation holds across both approaches and every configuration examined: the POS is never lower than the other three metrics among solved tasks, in Tables 6.2–6.5. For DreamCoder it stays strictly ahead of all three throughout; for DeepCoder it ties exactly with the PSS (Section 6.2.3), the closest the two can come without being the same metric. Both approaches, in other words, find the right operations at least as often as they arrange or edit them correctly, and DreamCoder specifically finds them measurably more often than it arranges them correctly. Figure 6.1 summarizes the solve rate together with all four token-based metrics for DeepCoder; Figure 6.2 does the same for DreamCoder's three configurations.

Of the four metrics, the POS tests the least strict condition: whether an operation

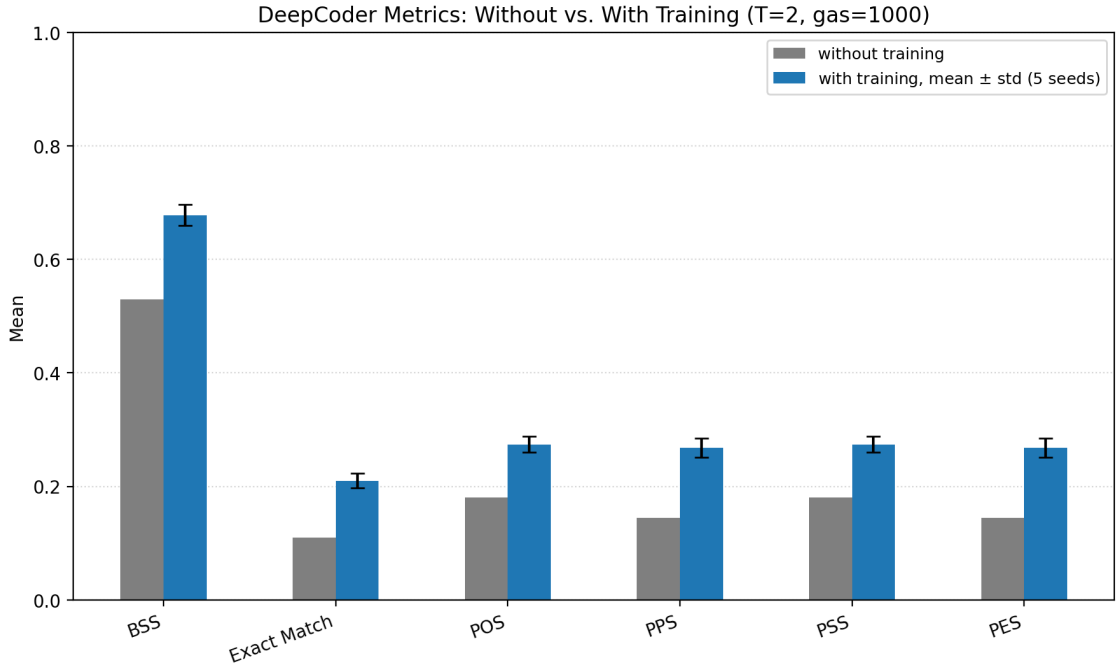

Figure 6.1: DeepCoder: solve rate, exact-match fraction, and the four token-based metrics, without vs. with training ($gas = 1{,}000$, mean over all 100 tasks).

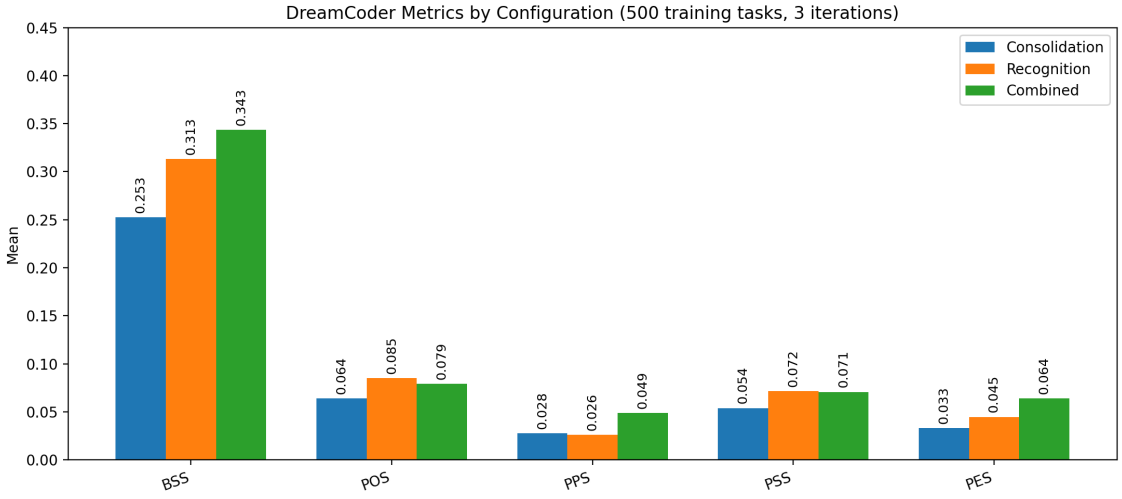

Figure 6.2: DreamCoder: solve rate and the four token-based metrics by configuration (500 training tasks, 3 iterations, mean over all 99 tasks).

occurs at all, regardless of where or in what order. That the POS is never lower than the other three metrics in any configuration follows already from exactly this weaker condition and the definitions of the four metrics, and holds for every individual program pair, not only on average. The PPS counts positions where both sequences match

exactly; every such position also constitutes a valid occurrence of the same token in both multisets, so the PPS can never count more matches than the multiset intersection on which the POS is based. The same conclusion holds for the PSS: the longest shared contiguous block uses, of any token involved, at most as many copies as actually occur in both sequences, and is therefore likewise bounded above by the multiset intersection. The same bound holds for the PES: in every optimal edit sequence between two programs, a set of positions remains unchanged, and its size can be at most as large as the multiset intersection, since every unchanged position is a matching token pair; because the PES is defined only in terms of these unchanged positions minus the insertions or deletions needed to bridge any length difference, the PES too can never exceed the POS. The POS can therefore never lie lower than the PPS, PSS, or PES for any program pair, regardless of how well or poorly a search actually performs.

That the POS is never lower is therefore not itself an empirical finding about DeepCoder or DreamCoder, since it is guaranteed by the metric definitions; what is empirical is how large the gap to the other three metrics turns out to be in each case, down to exactly zero in the limiting case of DeepCoder's PSS. A gap of zero between the POS and the PSS, as DeepCoder shows at this program length, means only that whatever token overlap the POS finds is always retained as a contiguous block under the PSS; it says nothing about whether that overlap sits in the reference's exact position, which the PPS measures separately and which stays a meaningfully positive gap throughout (Section 6.2.2). For DreamCoder, the POS–PSS gap is modest but consistent: most of the operation overlap captured by the POS is still retained as a contiguous block under the PSS. The substantially larger POS–PPS gap therefore indicates that exact position, rather than contiguity, is the more restrictive structural dimension in these runs. The remaining POS–PSS difference can partly be explained by DreamCoder solutions normalizing to sequences of varying length, which creates more opportunities for individual operation overlap without preserving the full contiguous structure. The previous sections found this gap to differ sharply in size, both between the two approaches and between metrics within DeepCoder itself. These differences are consistent with the training, search, and representation characteristics discussed earlier, rather than being explained by the metrics' guaranteed ranking alone, and it is the magnitude of these gaps that provides the informative signal.

Since the ranking itself is guaranteed, what the following three checks test is whether the gap behind it is robust rather than an artifact of how it was measured: whether it survives the zero convention for unsolved tasks, whether it extends beyond solved tasks

to unsolved ones with only partial progress, and whether it holds in comparable form for both approaches rather than being a quirk of just one.

6.3.2 All Tasks versus Solved Tasks

Across all four metrics and both approaches, the mean of each individual metric over solved tasks is consistently higher than its mean over all tasks: roughly one and a half to two times as high for DeepCoder, by a factor of three to four for DreamCoder. This follows directly from the convention that an unsolved task without a generated candidate enters every metric as 0; the mean over all tasks is therefore close to the mean over solved tasks multiplied by the solve rate, and reporting only the all-tasks view would understate how structurally close the search actually gets on the tasks it does solve. Restricting the analysis to solved tasks removes the influence of zero-valued unsolved tasks, while the observed gaps between POS and the other structural metrics remain clearly visible across configurations. Figure 6.3 isolates the four token-based metrics for DeepCoder over solved tasks only.

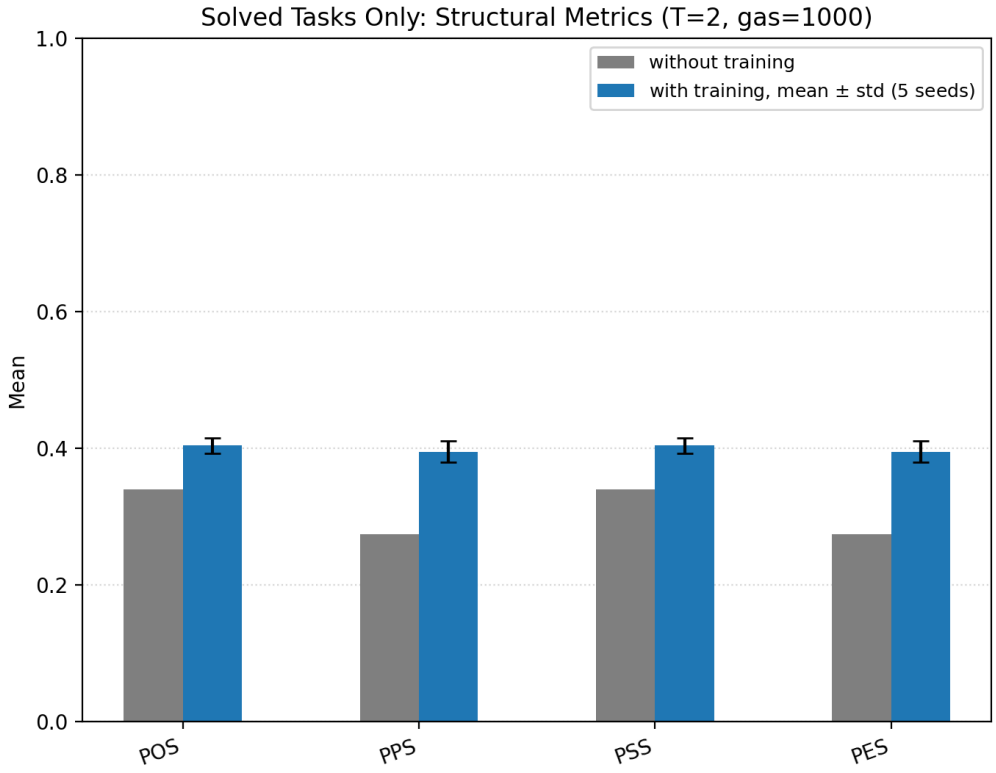

Figure 6.3: DeepCoder: the four token-based metrics without vs. with training, averaged over solved tasks only.

These different solve rates also mean different sample sizes behind the solved-only means. DeepCoder solves 53 to 70 of the 100 tasks depending on configuration, DreamCoder only 25 to 34 of the 99. DreamCoder's solved-only means therefore rest on markedly fewer data points than DeepCoder's, fewest for Consolidation with only 25 solved tasks. Comparisons among the three DreamCoder configurations on the solved side, such as the combined configuration's lower PSS, therefore rest on a smaller, statistically noisier sample than the corresponding DeepCoder comparisons, one reason to read small differences among the three DreamCoder configurations more cautiously than the considerably larger, more robust gap between the two approaches overall.

The solved-task analysis confirms the gap between POS and the other structural metrics. For unsolved tasks, however, the zero convention still hides how close the search came to a solution. For DeepCoder, this can be made more precise: the best-effort candidates already captured during the search make it possible to assign a value beyond 0 to part of the unsolved tasks as well.

6.3.3 Best-Effort Candidates on Unsolved Tasks

The view restricted to solved tasks is not the whole story on the unsolved side either. For unsolved tasks, DeepCoder's search additionally captures a *best-effort candidate*: the partial candidate that satisfied the most of a task's given examples during the search, even when no candidate solved the task fully. This is the extension to DeepCoder's search announced as a contribution of this thesis, and it makes it possible to compute the four token-based metrics for a subset of the unsolved tasks instead of uniformly returning 0. Because the configuration without training is deterministic, its 47 unsolved tasks and 25 stored best-effort candidates are identical across all five seeds; the solve rate of the configuration with training instead varies per seed, and so does the resulting set of unsolved tasks, which ranges from 30 to 34 tasks, of which 19 to 22 have a stored candidate. Not every unsolved task has a stored candidate at all, so this section reports two different views rather than one. Table 6.6 and Figure 6.4 average over *every* unsolved task, scoring the ones without a candidate as 0, the same zero-convention used for unsolved tasks throughout this chapter; Table 6.7 and Figure 6.5 restrict the same four metrics to only the tasks that actually have a stored candidate. The two answer different questions: the first asks how close the search gets across the unsolved side as a whole, folding in how often it keeps a candidate at all; the second asks, given that a candidate was kept, how close that specific candidate is to the reference.

Averaged this way, POS reaches only 0.021 without training and a mean of 0.043 ±

Table 6.6: Best-effort structural metrics averaged over all unsolved tasks in that seed (tasks without a stored candidate score 0; the count in parentheses is the number of those unsolved tasks with a stored candidate).

Configuration	Unsolved	POS	PPS	PSS	PES
Without training	47 (25)	0.021	0.000	0.021	0.000
With training, mean ± std	32.2 (20.2)	0.043 ± 0.011	0.003 ± 0.006	0.043 ± 0.011	0.003 ± 0.006

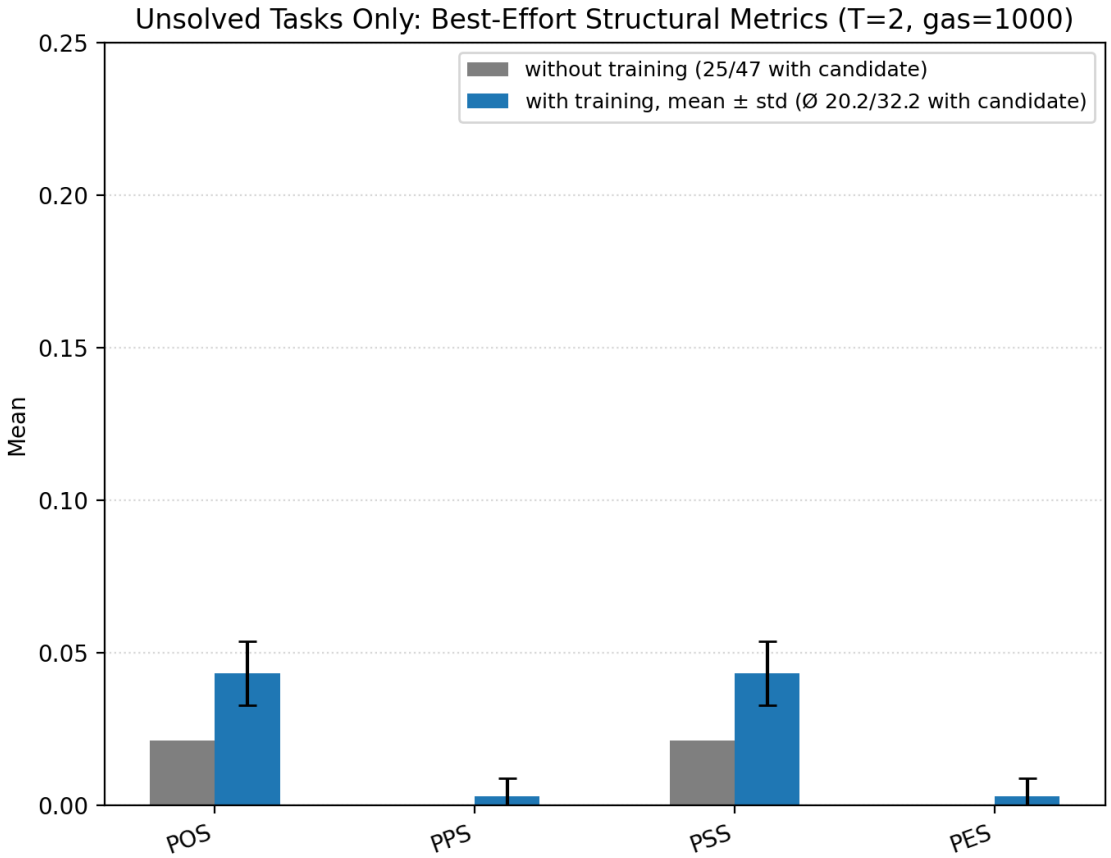

Figure 6.4: DeepCoder: the four token-based metrics averaged over all unsolved tasks, tasks without a stored candidate scored 0, without training vs. with training.

0.011 with training, and the PPS and PES stay close to 0 throughout (Table 6.6). Part of this low value simply reflects how many unsolved tasks have no candidate to score at all: 22 of the 47 without training, and between 10 and 14 with training, enter this average as a hard 0 regardless of how the search's actual best attempt on other tasks looked.

Restricting to only the tasks with a stored candidate removes the zero-inflation from

Table 6.7: Best-effort structural metrics restricted to unsolved tasks that have a stored candidate (the count matches the number in parentheses in Table 6.6).

Configuration	With candidate	POS	PPS	PSS	PES
Without training	25	0.040	0.000	0.040	0.000
With training, mean ± std	20.2	0.069 ± 0.016	0.005 ± 0.010	0.069 ± 0.016	0.005 ± 0.010

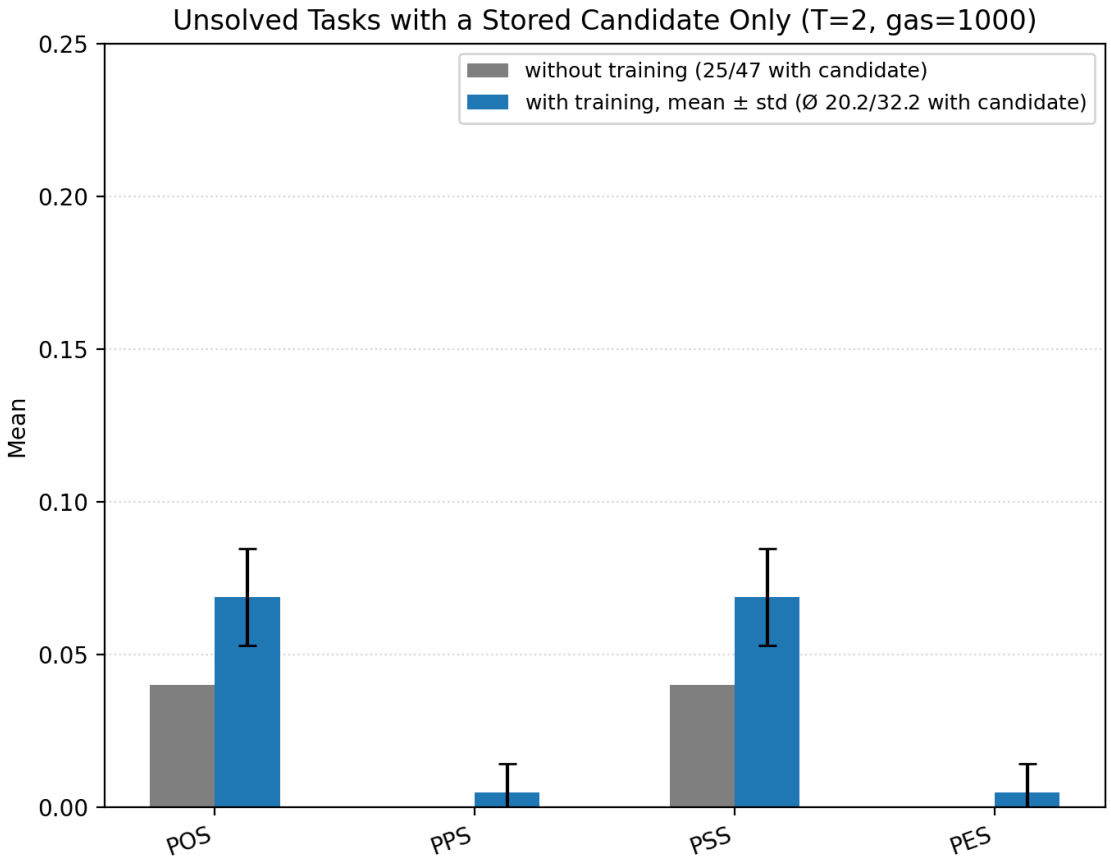

Figure 6.5: DeepCoder: the four token-based metrics averaged only over unsolved tasks with a stored best-effort candidate, without training vs. with training.

missing candidates, yet the values in Table 6.7 remain low: POS reaches no more than 0.040 without training and 0.069 ± 0.016 with training, and the PPS and PES again stay close to 0. So even a captured best-effort candidate, chosen specifically because it satisfied more of a task's given examples than any other candidate the search tried, shows only a small amount of measured structural overlap with the reference, and what little there is lies almost entirely in token presence and contiguity, POS and PSS, while exact positional agreement, PPS and PES, remains nearly absent. Satisfying more

given examples than any competing candidate is therefore a comparatively weak proxy for structural closeness: this is a different picture from what a near miss might suggest, a handful of scattered token differences from an otherwise correct program. A purely binary evaluation could not draw this distinction in the first place, since it does not track partial example-satisfaction on unsolved tasks at all.

Under either view, the absolute structural scores on unsolved tasks remain much smaller than on solved tasks, while most of the limited overlap is concentrated in operation presence and contiguity rather than exact position or edit similarity. DreamCoder has no comparable best-effort mechanism, so the cross-approach comparison is restricted to tasks solved by the respective approaches.

6.3.4 DeepCoder versus DreamCoder

Under their respective normalized representations, DeepCoder obtains higher numerical values than DreamCoder on all four metrics in every configuration, but these values should be treated as descriptive rather than as a representation-independent head-to-head quality ranking.

Figure 6.6 shows the same three DreamCoder configurations as Figure 6.2, now averaged over only the solved tasks instead of all 99, and without the BSS itself, which would by definition always equal 1 on this subset.

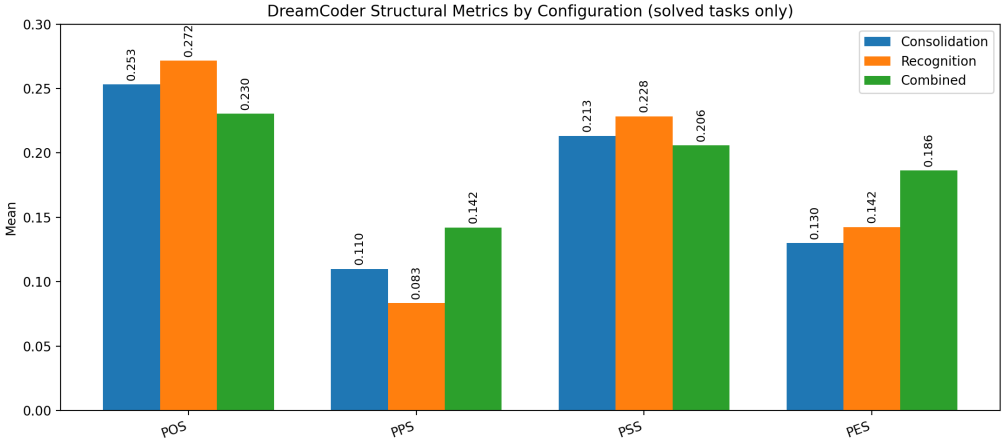

Figure 6.6: DreamCoder: the four token-based metrics by configuration, averaged over solved tasks only.

What differs between the two approaches is therefore not the guaranteed metric ordering, but the magnitude of the gaps behind it. DreamCoder shows a substantially stronger separation between operation presence and the stricter structural dimensions,

whereas DeepCoder exhibits smaller gaps and an exact coincidence between POS and PSS in the observed program pairs.

These aggregate numbers alone, however, say nothing about what a single pair of programs responsible for a particular POS, PPS, PSS, or PES value actually looks like. Three individual cases make this concrete.

6.3.5 Worked Examples

Each of the three cases below establishes its own point. The first shows that even an unsolved task can lie structurally close to the reference, something the BSS alone cannot express. The second shows the reverse case: a solved task whose program lies structurally far from the reference. The third shows why a perfect BSS can still say nothing about actual understanding.

For the DeepCoder task with reference program

`LIST|SCAN1L,*,0|ZIPWITH,max,1,0`

the leading `LIST` token only marks the type of the input parameter, here a single list, not its actual value; consistent with the convention introduced in Section 5.1, it is excluded from the token sequence the four metrics actually compare, leaving

`[SCAN1L,*,0, ZIPWITH,max,1,0]`

as the scored sequence. The reference program first computes the running product of the input list (`SCAN1L,*,0`), then replaces each running-product value with the larger of it and the corresponding original element (`ZIPWITH,max,1,0`), capping the running product from below by the original values wherever it would otherwise fall beneath them. Within the search budget, the search found no program that solved all five given examples. Its best candidate,

`LIST|HEAD,0|SCAN1L,*,0` $\rightarrow$ `[HEAD,0, SCAN1L,*,0]`,

computes the same running product, but the value `HEAD,0` extracts from the input is never used by the following `SCAN1L,*,0` call, which operates on the original input list again rather than on `HEAD,0`'s result; `HEAD,0` is therefore dead code, and the candidate reduces to the uncapped running product alone, without the reference's capping step.

Table 6.8 shows both programs on all five given examples: the candidate matches

Table 6.8: All five given examples of the task from the first example, with the output expected by the reference program and the actual output of the best candidate found.

No.	Input	Expected output (reference)	Found output (candidate)
1	-1, 0, -1, 0, 0, 0, 0, 0, -1, -1, 0, 0, 0, -1, 0, 0	-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0	-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
2	0, -1, -1, -1, 0, -1, 0, 0, 0, -1, -1, -1, 0, -1, -1, -1	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0	0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
3	12, -12	12, -12	12, -144
4	-1, 0, 0, 0, -1, -1, 0, -1, 0, 0, -1	-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0	-1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
5	5, 3, 1	5, 15, 15	5, 15, 15

the expected output on four of the five, examples 1, 2, 4, and 5, and diverges only on example 3, where the uncapped running product (-144) falls far below what the reference's capping step would have produced (-12), which is why the task overall remains unsolved despite coming this close behaviorally. On the token sequence actually scored, the candidate shares the `SCAN1L,*,0` operation with the reference, but at the opposite position, and drops the reference's `ZIPWITH,max,1,0` entirely in favor of an unused `HEAD,0` step:

$$\mathrm{POS} = \mathrm{PSS} = \frac{1}{2}, \qquad \mathrm{PPS} = \mathrm{PES} = 0.$$

POS and PSS agree because the one shared operation, wherever it sits, already forms a contiguous block on its own; PPS and PES agree because that same operation sits at a different position in each program, which costs the same whether counted as a wrong position or as a substitution needed to fix it. The task remains unsolved under the BSS, yet a candidate that matches four of five examples while sharing only half its operations, and none of them in position, shows that behavioral closeness and structural closeness can diverge sharply even on a near miss: an intermediate result no pure pass/fail judgment could express at all.

The second example shows the reverse case. For the DeepCoder task with reference program

`LIST|FILTER,>0,0|MAP,*-1,1` $\rightarrow$ `[FILTER,>0,0, MAP,*-1,1]`,

the reference program first filters out all positive elements and then negates the remaining

ones. The trained search instead found

$$\texttt{LIST|MAP,*-1,0|FILTER,<0,1} \quad \rightarrow \quad \texttt{[MAP,*-1,0, FILTER,<0,1]},$$

which reverses the order: it first negates every element and then keeps only the negative results, the same inversion by which $x > 0$ already becomes $-x < 0$.

Table 6.9: All five given examples of the task from the second example, with the output expected by the reference program and the actual output of the found solution.

No.	Input	Expected output (reference)	Found output (solution)
1	48, 242, -126, -107, -28	-48, -242	-48, -242
2	142, -78, -54, 60	-142, -60	-142, -60
3	19, 161, -74, 63, -114, -5, 40, -36, -243, 113, -52, -163, -8, -161, -241, -149, 141	-19, -161, -63, -40, -113, -141	-19, -161, -63, -40, -113, -141
4	28, -119, 113, 92, 78, 160, 130, -142, 152	-28, -113, -92, -78, -160, -130, -152	-28, -113, -92, -78, -160, -130, -152
5	103, 49, 251, -124, 161, 133, 74, -158, -40, -53, 140, -114, 159, 48, 31, -84, -8	-103, -49, -251, -161, -133, -74, -140, -159, -48, -31	-103, -49, -251, -161, -133, -74, -140, -159, -48, -31

Table 6.9 shows both programs on all five given examples: they produce the same, correct output in every single case, so the task counts as solved. Structurally, however, the two programs share nothing at all once the shared input-type token is set aside: swapping the order of `FILTER` and `MAP` also shifts their argument indices, so neither composite token the reference uses reappears anywhere in the solution:

$$\mathrm{POS} = \mathrm{PPS} = \mathrm{PSS} = \mathrm{PES} = 0.$$

A solved task and a program with zero measured structural overlap to the reference can therefore be the same task, exactly the case the BSS alone cannot distinguish from a solved task whose program stays close to the reference.

The third example is a case where the BSS is right for the wrong reason. For the task with reference program `LIST|LIST|INT|DROP,2,0|ZIPWITH,+,1,3`, whose three leading type markers `LIST`, `LIST`, and `INT` this time mark two lists and an integer as input parameters, and which drops the first two elements of the first list and adds the result element-wise to the second list, the expected output of all five given examples happens to be the empty list. DreamCoder found the program `(lambda (lambda (lambda (sort`

`empty))))`, which ignores its input entirely and simply returns a constant.

Table 6.10: All five given examples of the task from the third example. Since the found program ignores its input entirely, only the lengths of the two input lists and the integer are given here, not the actual values.

No.	Input (list A, list B, integer)	Expected output	Found output
1	16 elements, 19 elements, 37	empty list	empty list
2	7 elements, 9 elements, 34	empty list	empty list
3	14 elements, 4 elements, 228	empty list	empty list
4	7 elements, 15 elements, 239	empty list	empty list
5	5 elements, 19 elements, 58	empty list	empty list

Table 6.10 shows all five given examples: the inputs differ substantially in length and value, yet the expected output is the empty list in every case, exactly what the found program returns regardless of the actual input. Because the expected outputs of this particular task happen to coincide with what this constant program returns, it satisfies every given example and is recorded as solved with a BSS of 1, even though it does not implement the intended transformation at all. Such programs remain part of the BSS-defined solved set because they satisfy all five given examples, but they are flagged separately as *trivial solutions* for qualitative interpretation. They are rare but not unique: of the 99 test tasks, this affects the same four tasks in each of the three DreamCoder configurations, whose example sets likewise permit a constant output value. More input-output examples per task, or examples deliberately chosen so that a constant output is implausible, would make it harder for this particular kind of program to pass as a solution, but the underlying risk is more general: any evaluation based on a finite set of examples can in principle be satisfied by a program that ignores its input, and a larger example budget only reduces this risk rather than eliminating it.

Whether such a trivial solution is even noticeable in the token-based metrics is itself left to chance. For this comparison, reference and solution are represented in the shared normalized operation vocabulary, here without the position and parameter details retained by DeepCoder's native tokenization:

`DROP,ZIPWITH` → `[DROP, ZIPWITH]`, `SORT,EMPTY_CONST` → `[SORT, EMPTY_CONST]`.

Reference and found solution therefore share not a single token:

$$\text{POS} = \text{PPS} = \text{PSS} = \text{PES} = \frac{0}{2} = 0.$$

For the same kind of task, where the expected output happens to be the empty list throughout, the same combined DreamCoder configuration found the equally trivial solution `(lambda (#(map (lambda (negate $0))) empty))` for a different reference, `LIST|MAP,**2,0|FILTER,<0,1`, which just as before operates exclusively on the constant `empty` and ignores its input entirely. Tokenized:

`MAP,FILTER` → `[MAP, FILTER]`, `MAP,EMPTY_CONST` → `[MAP, EMPTY_CONST]`.

Table 6.11: All five given examples of the second trivial task.

No.	Input	Expected output	Found output
1	11, -3, 7, 11, -8, 1, 5, -12, -1, 14, 7, -7, -4, 0, 9	empty list	empty list
2	6	empty list	empty list
3	0, 15, -12, 9, 15, -14, 3, -5, 9, -3, -6, -15, -4	empty list	empty list
4	11, -16, 4, 4, 15, 7, 13, 13, -10, 3, 15, 6, 7, -12, 14, 5	empty list	empty list
5	15, -2, 0, 14, 4, 15, -14, -4	empty list	empty list

Table 6.11 shows that here too, all five examples happen to expect the empty list regardless of the input's length or values, so the input-ignoring program satisfies every example as well. This time, both programs happen to share the first token, `MAP`, the same operation family as the reference, even though this solution also never looks at its input:

$$\text{POS} = \text{PPS} = \text{PSS} = \text{PES} = \frac{1}{2}.$$

Two equally meaningless solutions thus receive different metric values, 0 in the first case, 0.5 in the second, solely depending on whether they happen to share a surface token with their respective reference, not on whether they actually compute anything meaningful.

This third case makes visible that the tools used in this chapter, BSS and token-based metrics alike, run into a fundamental limit as soon as a program satisfies the given examples without implementing the intended rule. Which further limitations the above results have is summarized in the last section of this chapter.

6.4 Limitations

Five limitations qualify how the results above should be read.

Structural closeness is not semantic equivalence. None of the four metrics constitutes a test of semantic equivalence; they measure structural closeness to one specific reference program, not whether two programs compute the same function for all possible inputs. Two structurally very different programs can still be functionally identical, which the token-based metrics cannot establish because they compare program structure rather than semantic behavior, precisely the situation in the second example above, where a solved task computing a valid, differently structured solution scored zero on all four metrics. The asymmetry also runs the other way: a single substituted token can already change the computed function on most inputs while barely moving any of the four metrics, so a high value is no more a guarantee of semantic correctness than a low value is a guarantee of semantic difference. The trivial-solution case among the examples above pushes the same gap to a more fundamental extreme still: unlike the second example's legitimate alternative algorithm, which merely happens to share no tokens with the reference, a program that never looks at its input can still achieve a perfect BSS if it reproduces all given examples. The BSS evaluates behavioral correctness on these examples, whereas the token-based metrics evaluate structural similarity to the reference program. Neither perspective, however, establishes that the generated program implements the intended semantics for all possible inputs. Additional held-out examples could provide stronger behavioral evidence, but they would reduce rather than eliminate this uncertainty.

The DreamCoder mapping is incomplete. Comparing DeepCoder and DreamCoder at all requires mapping DreamCoder's lambda-calculus expressions onto a shared normalized operation vocabulary centered on the DeepCoder operation families, and this mapping only covers operation families for which a correspondence was defined; an operation expressed in a way that was not anticipated remains unmapped. DeepCoder's native tokens retain each operation's exact parameters and variable indices, while DreamCoder's mapped representation primarily records operation families together with a small number of normalization-specific abstract categories. DreamCoder's programs are therefore represented more coarsely than DeepCoder's natively tokenized ones. Part of the gap reported above between the two approaches therefore reflects the measurement process itself rather than a genuine difference in how close either approach gets to the reference.

A more exhaustive mapping, covering additional operation families and parameter-level detail, could narrow this gap without any change to either approach's underlying search.

The evaluation covers one domain and one reference-program length. All results in this chapter concern list-processing tasks whose reference programs contain exactly two chained DeepCoder operations. This restriction applies to the reference programs, not to all generated solutions: in particular, DreamCoder solutions may normalize to shorter or longer token sequences. Longer reference programs, other domains, or datasets with a different distribution of operations could therefore change both the solve rates and the magnitudes of the gaps between the four structural metrics. The fact that the POS upper-bounds the PPS, PSS, and PES, however, follows from the metric definitions and is independent of program length. For DeepCoder, the two-operation setting restricts the possible overlap patterns and, together with the overlap patterns observed in the evaluated program pairs, contributes to the observed equality between POS and PSS. DreamCoder, by contrast, already shows small POS–PSS differences because its generated programs can normalize to sequences of varying length and because its representation is structurally different. More generally, any program-level comparison depends on how the compared approaches represent and normalize programs in the first place. A different pair of approaches, or a different normalization scheme, could therefore weaken or strengthen the observed metric gaps.

The cross-approach comparison is descriptive rather than compute-matched. DeepCoder and DreamCoder use fundamentally different search-budget units and training procedures, so their absolute solve rates and structural scores should not be interpreted as establishing which approach is intrinsically superior. The purpose of evaluating both approaches is instead to examine whether the proposed metrics remain applicable and informative across different synthesis paradigms.

The two approaches are reported with different reproducibility guarantees. DeepCoder's trained results are aggregated across five independent seeds, whereas each DreamCoder configuration is represented by a single run. DreamCoder's run-to-run variance therefore remains unknown, and part of an observed cross-approach difference could in principle reflect variance that cannot be quantified for DreamCoder.

7 Conclusion

This thesis set out from the question of what additional insight into the performance of PBE approaches can be gained through evaluation metrics beyond binary success signals. To answer this question, four token-based metrics were developed, POS, PPS, PSS, and PES, which compare not an approach's output but the program it generates itself to a reference program, and applied to two structurally fundamentally different approaches: DeepCoder, which searches over a fixed DSL, and DreamCoder, which extends its own library during the search. Because a direct token comparison between DeepCoder's DSL programs and DreamCoder's lambda-calculus expressions is not possible without further work, this first required mapping both program representations onto a shared normalized operation vocabulary.

The four metrics capture complementary dimensions of structural similarity: operation presence, exact position, contiguous structure, and edit effort. Read together with the BSS, they provide a more differentiated view of program behavior than either could on its own.

Across five configurations examined, DeepCoder without and with a trained attribute predictor over five seeds (solve rate from 53.0% to 67.8%$\pm$1.8%) as well as DreamCoder's three learning mechanisms Consolidation, Recognition, and their combination (solve rate between 25.3% and 34.3%), a consistent pattern emerged: the POS was, without exception, never lower than the other three metrics, a ranking guaranteed by the metrics' definitions rather than an empirical finding in itself. Both approaches, in other words, find the right building blocks of a solution at least as often as they arrange those building blocks correctly. What is empirical is that the resulting gap held up under three further views: the stricter view of solved tasks only, DeepCoder's best-effort candidates on tasks no search fully solved, and DreamCoder just as much as DeepCoder, even though the two approaches differed in practically every other respect.

That a program solves the given examples rarely means it matches the reference: only between 11.0% and 21.0% of all DeepCoder tasks were matched exactly, token for token, even though considerably more count as solved. Three worked individual cases made

visible what this means in practice. A best-effort candidate that matched four of the five given examples nonetheless shared only half of the reference's operations, and none of them in position, a gap between behavioral and structural closeness a pure pass/fail judgment cannot express. A solved DeepCoder task, conversely, scored zero on all four metrics, because its solution produced the same output by a route with zero measured composite-token overlap with the reference. A third example finally showed the limit of both views at once: a program that ignored its input entirely happened to satisfy the given examples and thereby received a perfect BSS, even though it did not implement the intended transformation.

A BSS conceals how close a wrong solution comes to the reference and whether a solved task is satisfied by a structurally close or structurally distant program. Program-level metrics close this gap by showing how much structural overlap with the reference is present and in what respect a solution deviates from it, in the presence of the right operations, in their position, or in their contiguity, and whether the same kind of deviation recurs across configurations and even across structurally different approaches. They are most informative precisely where the BSS distinguishes least: on unsolved tasks with partial progress, and on solved tasks whose solution deviates structurally from the reference.

These findings hold within the limitations named in the previous chapter: they rest on a single domain and a length restriction to two operations, on an incomplete mapping between DreamCoder's and DeepCoder's program representations, on structural rather than semantic closeness, and on asymmetric reproducibility guarantees, with DeepCoder's trained results averaged across five seeds while each DreamCoder configuration is represented by a single run.

7.1 Future Work

These limitations also point to natural directions for future work. Most immediately, one could test how the observed gaps between POS and the stricter structural metrics change beyond the list-processing domain and the two-operation length restriction examined here, for instance in the text domain or with longer reference programs, where more room exists for reordering and partial correctness.

A more exhaustive normalization of DreamCoder's lambda-calculus expressions, covering additional operation families and parameter-level detail, could further reduce representation-induced differences between the two approaches. Likewise, the best-effort

mechanism introduced for DeepCoder in this thesis could be transferred to DreamCoder, to make the insights gained on the unsolved side available for both approaches rather than only one.

Finally, future work could combine the token-based metrics developed in this thesis with additional behavioral validation on held-out inputs to assess whether structurally different programs also behave equivalently beyond the given examples. Such testing would provide stronger behavioral evidence without by itself establishing semantic equivalence.

References

[1] Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017. URL `https://arxiv.org/abs/1611.01989`.

[2] José Cambronero, Sumit Gulwani, Vu Le, Daniel Perelman, Arjun Radhakrishna, Clint Simon, and Ashish Tiwari. FlashFill++: Scaling programming by example by cutting to the chase. *Proc. ACM Program. Lang.*, 7(POPL):33, 2023.

[3] Mark Chen, Jerry Tworek, et al. Evaluating large language models trained on code, 2021.

[4] Fred J. Damerau. A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3):171–176, 1964.

[5] Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel-rahman Mohamed, and Pushmeet Kohli. RobustFill: Neural program learning under noisy I/O. In *Proceedings of the 34th International Conference on Machine Learning*, 2017. URL `https://arxiv.org/abs/1703.07469`.

[6] Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Joshua B. Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a REPL. In *33rd Conference on Neural Information Processing Systems (NeurIPS 2019)*, 2019.

[7] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep bayesian program learning, 2020. URL `https://arxiv.org/abs/2006.08381`.

[8] Kevin Ellis et al. ec (official dreamcoder implementation). GitHub repository. URL `https://github.com/ellisk42/ec`.

[9] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '11)*, 2011.

[10] Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 2017.

[11] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *1st Mathematical Reasoning in General Artificial Intelligence Workshop, ICLR 2021*, 2021. URL `https://github.com/hendrycks/apps`.

[12] David Kamm. deepcoder (unofficial deepcoder reference implementation). GitHub repository. URL `https://github.com/dkamm/deepcoder`.

[13] Maurice G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1/2):81–93, 1938. doi: 10.2307/2332226.

[14] Tessa A. Lau, Steven A. Wolfman, Pedro Domingos, and Daniel S. Weld. Programming by demonstration using version space algebra. *Machine Learning*, 53:111–156, 2003.

[15] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

[16] Wen-Ding Li and Kevin Ellis. Is programming by example solved by LLMs? In *38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024.

[17] Chin-Yew Lin and Franz Josef Och. Automatic evaluation of machine translation quality using longest common subsequence and skip-bigram statistics. In *Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics*, 2004. doi: 10.3115/1218955.1219032.

[18] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002. doi: 10.3115/1073083.1073135.

[19] Oleksandr Polozov and Sumit Gulwani. FlashMeta: A framework for inductive program synthesis. In *Proceedings of the ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, 2015. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/oopsla15-pbe.pdf.

[20] Oona Rainio, Jarmo Teuho, and Riku Klén. Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14:6086, 2024. doi: 10.1038/s41598-024-56706-x.

[21] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. CodeBLEU: a method for automatic evaluation of code synthesis, 2020. URL https://arxiv.org/abs/2009.10297.

[22] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020. URL https://aclanthology.org/2020.acl-main.442/.

[23] Tobias Sesterhenn, Ian Berlot-Attwell, Janis Zenkner, and Christian Bartelt. A compute-matched re-evaluation of TroVE on MATH. In *The Second AI for MATH Workshop at ICML 2025*, 2025.

[24] Kensen Shi, Joey Hong, Yinlin Deng, Pengcheng Yin, Manzil Zaheer, and Charles Sutton. ExeDec: Execution decomposition for compositional generalization in neural program synthesis. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.13883.

[25] Yewei Song, Cedric Lothritz, Daniel Tang, Tegawendé F. Bissyandé, and Jacques Klein. Revisiting code similarity evaluation with abstract syntax tree edit distance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2024. URL https://aclanthology.org/2024.acl-short.3/.

[26] Bo Wang, Teodora Baluta, Aashish Kolluri, and Prateek Saxena. SynGuar: Guaranteeing generalization in programming by example. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021. doi: 10.1145/3468264.3468621.

[27] Anjiang Wei, Tarun Suresh, Jiannan Cao, Naveen Kannan, Yuheng Wu, Kai Yan, Thiago S. F. X. Teixeira, Ke Wang, and Alex Aiken. CodeARC: Benchmarking reasoning capabilities of LLM agents for inductive program synthesis, 2025. URL https://arxiv.org/abs/2503.23145.

[28] Steve Leonel Yomi Mbiakop. deepcoder (fork with the extensions and evaluation pipeline developed for this thesis). GitHub repository, 2026. URL https://github.com/YOMILEONEL/deepcoder.

[29] Steve Leonel Yomi Mbiakop. ec (fork with the extensions and evaluation pipeline developed for this thesis). GitHub repository, 2026. URL https://github.com/YOMILEONEL/ec.

[30] Janis Zenkner, Tobias Sesterhenn, and Christian Bartelt. Shedding light on task decomposition in program synthesis: The driving force of the synthesizer model. In *Published as a workshop paper at ICLR 2025*, 2025. URL https://arxiv.org/abs/2503.08738.

[31] Jiasheng Zheng, Boxi Cao, Zhengzhao Ma, Ruotong Pan, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. Beyond correctness: Benchmarking multi-dimensional code generation for large language models, 2024. URL https://arxiv.org/abs/2407.11470.